Rapport de PFE : Évaluation de la robustesse des LLMs face à une documentation contrefactuelle

Maximilien Lyonnais, Clément Géliot, Hugo De Bosschere

8 avril 2026

Résumé

Retrieval-Augmented Generation (RAG) est une technique consistant à extraire des documents pertinents depuis une base de données externe pour les injecter directement dans la fenêtre de contexte du modèle avant l'étape de génération. Les grands modèles de langage (Large Language Models ou LLMs) s'appuient de plus en plus sur des systèmes de RAG pour exploiter des informations récentes ou spécifiques. Cette approche met en lumière une tension fondamentale entre deux sources de connaissances : la *mémoire paramétrique*, qui correspond au savoir figé dans les poids du modèle lors de son pré-entraînement et la *mémoire contextuelle* qui représente ces nouvelles données injectées dans le prompt via le RAG lors de l'inférence. Un conflit émerge lorsque ces deux mémoires se contredisent, posant le problème de l'arbitrage : comment le modèle choisit-il la source d'information à privilégier ?

Dans ce rapport, nous étudions cette dynamique à travers un cas d'usage critique pour le développement logiciel : la mise à jour d'APIs. Nous simulons une situation où un LLM, entraîné sur une version antérieure d'une bibliothèque, reçoit en contexte la documentation de sa nouvelle version. Nous supposons donc ici un système RAG parfait capable de retrouver et de fournir exactement le bon document. Le comportement que nous souhaiterions dans ce cas d'usage est que le modèle privilégie systématiquement le contexte. Notre objectif est de mesurer rigoureusement la capacité du LLM de privilégier la mémoire contextuelle lors d'un conflit de cette nature. À travers une pipeline d'évaluation dédiée, nous quantifions la propension des modèles à rejeter leur mémoire paramétrique au profit de la mémoire contextuelle et ce en faisant varier de multiples configurations : l'architecture des modèles, leur nombre de paramètres, les jeux de données évalués, ainsi que la nature et l'intensité des perturbations simulées sur la bibliothèque.

Table des matières

1 Introduction **3**
1.1 Contexte général : Les limites des LLMs et l'essor du RAG . . . 3
1.2 Problème : Le conflit entre mémoire paramétrique et mémoire contextuelle . . . 3
1.3 Objectifs et cadre de l'étude : Le cas de la mise à jour d'APIs . . . 4
1.4 Contributions . . . 4

2 Revue de la littérature **4**
2.1 Évaluation de la génération de code par les LLMs . . . 4
2.2 Vulnérabilités contextuelles dans les systèmes RAG . . . 5

3 Méthodologie et Stratégie de Recherche **5**
3.1 Création de la documentation contrefactuelle . . . 6
3.1.1 Type de documentation . . . 6
3.1.2 Type de perturbation . . . 7
3.2 Inférence . . . 9
3.2.1 Moteur d'inférence . . . 9

3.2.2 Modèles testés : . . . 9
3.2.3 Type de tests réalisés : . . . 9
3.2.4 Hyperparamètres : . . . 10
3.2.5 Extraction et nettoyage du code généré . . . 10
3.2.6 Prompts : . . . 11
3.2.7 Aparté sur le pass@k . . . 11
3.3 Évaluation Fonctionnelle . . . 12
3.3.1 Mécanisme des Wrappers . . . 12
3.3.2 Nettoyage AST et double évaluation . . . 13
3.3.3 Environnement d'exécution . . . 13

4 Résultats et Analyse **14**
4.1 La famille de modèles qwen3.5 . . . 14
4.1.1 Première analyse . . . 14
4.1.2 Quelques définitions pour la suite . . . 15
4.1.3 Retour à l'analyse . . . 16
4.2 Echecs . . . 17
4.3 Analyse sur Codestral, Devstral, Gemma3:12b et Qwen2.5-Coder:32b . . . 20
4.3.1 Cadre expérimental et hyperparamètres . . . 20
4.3.2 Résultats avec la documentation explicative . . . 20
4.3.3 Résultats avec les documentations minimale et ultra-minimale . . . 22
4.3.4 Analyse de l'intelligence des modèles via les métriques de dégradation . . . 22
4.4 Documentation bruitée . . . 25
4.5 Contribution au benchmarking . . . 25

5 Difficultés et Limites **25**
5.1 Complexité méthodologique . . . 25
5.1.1 Modifications de DS1000 pour maintenir la cohérence . . . 25
5.1.2 Complexité du nettoyage AST . . . 26
5.1.3 Gestion du cas `ObjectAttributeError` . . . 26

6 Limitations et Ouvertures **26**
6.1 Limites de l'étude . . . 26
6.2 Ouvertures et Perspectives de recherche . . . 27

7 Conclusion **28**

8 Bibliographie **29**

A Annexes **31**
A.1 Graphiques . . . 31
A.2 Exemples de réponses LLM . . . 31
A.3 Configuration et Prompts . . . 33
A.4 Visualisation des différentes documentations . . . 33
A.5 Extraits de Code . . . 38

1 Introduction

1.1 Contexte général : Les limites des LLMs et l'essor du RAG

Les LLMs ont démontré des capacités remarquables dans une multitude de tâches, allant de la compréhension du langage naturel à la génération de code complexe. Cependant, ces modèles souffrent de limitations inhérentes à leur conception : ils sont sujets aux hallucinations [19, 22, 1] et demeurent strictement contraints par les connaissances figées dans leur corpus d'entraînement. Ils sont ainsi incapables, par nature, de répondre de manière fiable à des requêtes portant sur des événements récents ou des informations privées.

Pour pallier ce problème, la technique du *Retrieval-Augmented Generation* s'est imposée comme un standard de l'industrie. Le RAG consiste à extraire des documents pertinents depuis une base de données externe pour les injecter directement dans la fenêtre de contexte du modèle avant l'étape de génération [13, 3]. La plupart des LLMs commerciaux grand public, tels que ChatGPT [18] ou Gemini [9], intègrent déjà les mécanismes de recherche web via du RAG.

Bien que cette approche améliore considérablement la précision des modèles, elle soulève de nouveaux défis éthiques et sécuritaires lorsque les documents récupérés contiennent des informations erronées ou malveillantes [6]. Des exemples récents de modèles générant des recommandations absurdes illustrent les dangers d'une confiance aveugle accordée aux informations récupérées par le mécanisme de RAG. Un filtrage plus strict des documents ne suffit pas ; le véritable enjeu réside dans la capacité du modèle à arbitrer intelligemment les informations qu'il traite.

1.2 Problème : Le conflit entre mémoire paramétrique et mémoire contextuelle

Cette dynamique met en lumière une tension fondamentale au cœur de l'architecture des LLMs, partagés entre deux sources de connaissances distinctes :

- **La mémoire paramétrique** : le savoir encodé de manière statique dans les poids du réseau de neurones lors de sa phase de pré-entraînement.
- **La mémoire contextuelle** : les informations éphémères injectées directement dans le prompt (via le RAG ou l'utilisateur) lors de l'inférence.

Un conflit majeur émerge lorsque ces deux mémoires se contredisent. Dans une telle situation, comment le modèle choisit-il la source d'information à privilégier ? Un modèle trop "sûr de lui" ignorera systématiquement le contexte externe pour se raccrocher à ses croyances internes, générant ainsi des réponses obsolètes malgré la présence de la bonne information. À l'inverse, un modèle trop "docile" se laissera manipuler par n'importe quelle information ou ensemble d'informations contextuel, même factuellement faux ou incohérente.

Plusieurs travaux se sont penchés sur la nature de cette tension. Longpre et al. [15] ont démontré que les LLMs affichent une forte préférence pour les informations issues de leurs données d'entraînement, même lorsque des faits substitués (et contrefactuels) leur sont fournis dans le contexte. Plus récemment, Xie et al. [24] ont montré que la susceptibilité des modèles au contexte, ou au contraire leur biais vers leurs connaissances *a priori*, dépend fortement de la cohérence du contexte. Si celui-ci est cohérent le LLM aura tendance à accepter les nouvelles informations comme vraies et à les utiliser. Si le contexte contient des informations en accord et en désaccord avec sa mémoire paramétrique le LLM aura tendance à préférer les informations qui confirment ses croyances *a priori*. Dans cette lignée, les travaux de Tang et al. avec le benchmark *ClashEval* [23] proposent de quantifier précisément ce « bras de fer » (*tug-of-war*) entre les connaissances internes du modèle et les preuves externes. Ils démontrent ainsi que la propension du modèle à suivre le contexte est intimement liée à sa confiance interne (mesurable via les probabilités de tokens, ou *logprobs* [17, 25]) et au degré de déviation de l'information perturbée.

1.3 Objectifs et cadre de l'étude : Le cas de la mise à jour d'APIs

Notre étude prolonge ces travaux en explorant ce conflit cognitif à travers un cas d'usage précis : le développement logiciel, et plus particulièrement l'adaptation aux mises à jour d'APIs.

Dans un environnement de développement réel, les bibliothèques logicielles évoluent constamment. Un LLM déployé comme assistant de code a nécessairement été entraîné sur une version antérieure d'une bibliothèque. S'il reçoit en contexte la documentation de la nouvelle version mise à jour, un système RAG idéal — capable de retrouver et de fournir exactement le bon document d'API — est supposé garantir la génération d'un code valide. Dans ce scénario précis, le comportement attendu et souhaité est que le modèle privilégie *systématiquement* le contexte par rapport à ses habitudes d'entraînement. Cependant, les LLMs ne sont généralement pas entraînés pour accorder une confiance aveugle à leur contexte, en particulier lorsque celui-ci contredit des connaissances massivement ancrées dans leurs poids. Par conséquent, nous pouvons nous attendre à ce que plus la perturbation s'écarte des conventions établies, moins le modèle l'adoptera. Par exemple, une mise à jour fictive imposant une majuscule initiale à toutes les fonctions `NumPy` constitue une corruption syntaxique assez forte en Python. Il est possible qu'elle soit partiellement ignorée par le modèle, ce dernier préférant se replier sur sa mémoire paramétrique.

Notre objectif est donc de mesurer rigoureusement cette capacité d'adaptation de manière contrôlée et reproductible. Nous expliquons plus en détail notre méthodologie dans cette section. 3

1.4 Contributions

Notre projet s'inscrit dans la filiation directe de *ClashEval*, mais en transpose le paradigme abstrait (Questions-Réponses en langage naturel) vers le domaine concret et déterministe de l'ingénierie logicielle (Génération de code). En générant des documentations de bibliothèques contrefactuelles (minimales, ultra-minimales, explicatives, bruitées), nous appliquons des perturbations contrôlées comparables à celles de *ClashEval*. Cependant, notre méthode d'évaluation dépasse l'analyse textuelle : grâce à notre pipeline de nettoyage syntaxique et à nos *wrappers*, nous ne nous contentons pas d'évaluer la proximité sémantique d'une réponse, mais nous validons *fonctionnellement* l'arbitrage du modèle par l'exécution stricte du code généré.

2 Revue de la littérature

Notre étude se situe à l'intersection de deux domaines de recherche majeurs en traitement du langage naturel : l'évaluation des capacités de génération de code des grands modèles de langage, et l'étude des conflits cognitifs inhérents aux systèmes de *Retrieval-Augmented Generation* (RAG).

2.1 Évaluation de la génération de code par les LLMs

L'évaluation des LLMs sur des tâches de programmation a connu un essor important depuis 2021, avec l'apparition de benchmarks devenus aujourd'hui des standards de l'industrie.

Les travaux fondateurs de Chen et al. [5] ont introduit **HumanEval**, un ensemble de 164 problèmes Python accompagnés de tests unitaires. Ce travail a également démocratisé l'estimateur non-biaisé du *pass@k*, que nous utilisons directement dans notre pipeline d'évaluation pour quantifier de manière probabiliste les succès des modèles. MBPP (*Mostly Basic Programming Problems*) [2] est venu compléter cette approche avec près de 1000 problèmes de niveau débutant. Cependant, Liu et al. [14] ont démontré avec le framework **EvalPlus** que la faible couverture des tests unitaires dans HumanEval et MBPP conduisait à de nombreux faux positifs (un code syntaxiquement valide mais logiquement faux passant les tests). En multipliant la

taille des suites de tests, EvalPlus a révélé que les scores habituellement rapportés surestimaient significativement les performances réelles des modèles.

C'est pour pallier ces limites et évaluer des cas d'usage plus réalistes que Lai et al. [12] ont proposé **DS-1000**. Contrairement aux problèmes algorithmiques purs, DS-1000 se concentre sur des tâches de science des données extraites de questions réelles de *StackOverflow*, couvrant sept bibliothèques majeures. Pour éviter que les modèles ne se contentent de restituer des solutions mémorisées, DS-1000 introduit des mécanismes de perturbation de surface (renommage de variables) et sémantiques (modification des contraintes).

Notre cadre d'évaluation repose fondamentalement sur l'approche de DS-1000. Pour des raisons de temps de calcul et de profondeur d'analyse, nous avons extrait et isolé les 159 problèmes spécifiques à la bibliothèque NumPy pour former notre corpus de base. L'utilisation de **DS-1000** nous permet de tester les LLMs sur une API qu'ils ont massivement rencontrée durant leur entraînement, maximisant ainsi l'ancrage de leur mémoire paramétrique. Par ailleurs, alors que des initiatives comme le *BigCode Evaluation Harness* [16] proposent des évaluations unifiées statiques, ou que *SWE-bench* [10] évalue la navigation dans de larges dépôts de code, notre pipeline d'évaluation s'en distingue par la nécessité d'une *double exécution dynamique* (code testé simultanément contre une librairie réelle et une librairie artificiellement perturbée).

2.2 Vulnérabilités contextuelles dans les systèmes RAG

Bien que les LLMs soient performants, ils sont sujets aux hallucinations [11]. Les architectures RAG ont été largement adoptées pour atténuer ce phénomène en fournissant un contexte externe fiable [21]. En réponse à cette adoption, de nombreux frameworks d'évaluation automatisés pour le RAG, tels que RAGAS [7] ou ARES [20], ont vu le jour.

Néanmoins, les recherches récentes soulignent que le RAG est une arme à double tranchant (*double-edged sword*) [8]. Comme le démontrent Chen et al. [3], les modèles peuvent être induits en erreur par des résultats de recherche complexes ou trompeurs. Pire encore, dans le domaine médical, il a été prouvé que l'injection de contexte pouvait amener des modèles à générer des recommandations critiques erronées [1], confirmant que les modèles peinent souvent à intégrer correctement l'information externe, même lorsqu'elle est fournie explicitement.

Cette vulnérabilité des systèmes RAG nous amène au cœur de notre problématique : le conflit entre ce que le modèle « sait » (poids d'entraînement) et ce qu'on lui « dit » (contexte). C'est ce que nous allons tenter de quantifier maintenant.

3 Méthodologie et Stratégie de Recherche

Notre but était donc de mesurer et d'évaluer à quel point les LLMs sont capables de s'adapter à des informations qui ne sont pas seulement nouvelles mais qui sont en conflit direct avec les données sur lesquels ils ont été entraînés. Pour mesurer cette capacité d'adaptation et, le cas échéant, la perte de performance liée à cette nécessité d'adaptation nous avons tout d'abord généré des documentations contrefactuelles de trois types (ultra-minimale, minimale et explicative). Ensuite nous avons fourni cette documentation au LLM via le moteur d'inférence llama.cpp et le wrapper haut-niveau ollama. Nous avons fourni au LLM la documentation directement dans sa fenêtre de contexte (context window) en s'assurant que la taille de la documentation était inférieure à la taille maximale de la fenêtre de contexte habituelle du LLM. Nous avons utilisé le benchmark DS1000 en tant que base de problèmes à résoudre en ne gardant que les problèmes liés à la librairie NumPy (ce qui fait un total de 159 problèmes à résoudre) et en modifiant les énoncés des problèmes pour s'assurer que la modification NumPy y était présente aussi. Ensuite nous avons récupéré le code généré par le LLM via des méthodes de parsing. Finalement, nous avons fait de l'évaluation fonctionnelle du code généré pour nous assurer de deux choses : Premièrement que le code utilise bien les modifications censées correspondre à une mise à jour d'une

librairie. Deuxièmement que le code qui utilise les modifications idoines fonctionne correctement et renvoie les bonnes valeurs sur les différents test case de DS1000.

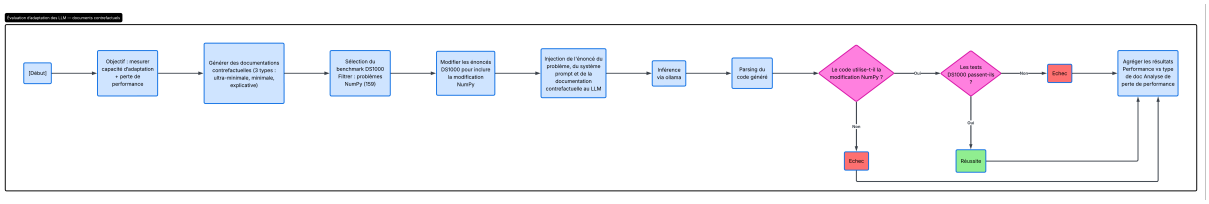

FIGURE 1 – Pipeline de travail, de la conception de la documentation contrefactuelle à l'évaluation fonctionnelle du code généré par le LLM

3.1 Création de la documentation contrefactuelle

La première étape de notre projet a donc été la création d'une documentation contrefactuelle à partir des docstrings existants. Pour ne pas surcharger la fenêtre de contexte nous avons choisi de nous concentrer sur une seule librairie qui devait forcément être dans les données d'entrainement du LLM et c'est pourquoi nous avons choisi d'utiliser la très usitée librairie `NumPy`. Pour générer la documentation de la librairie nous avons utilisé un crawler qui parcourt toutes les fonctions de la libraire numpy et récupère la partie du docstring qui nous intéresse en fonction du type de documentation que nous sommes en train de générer. La génération de la documentation est donc déterministe et ne fait pas intervenir un LLM pour appliquer la perturbation souhaitée. Nous avions tout d'abord choisi de générer cinq types de documentations à la fois factuelle et contrefactuelle :

3.1.1 Type de documentation

- **Documentation ultra-minimale :** Ce type de documentation catalogue les noms des différentes fonctions.
- **Documentation minimale :** La même chose que la documentation ultra_minimale en ajoutant en plus les arguments pris par la fonction et la signature de la fonction quand celle-ci existe dans le docstring.
- **Documentation explicative :** Explication de la perturbation. Ce type de documentation s'éloigne un peu du champ de notre recherche puisque le but final est de pouvoir donner au LLM la documentation de la nouvelle librairie et que celui-ci sans serve sans avoir besoin de lui dire ce qui a changé par rapport à la version sur laquelle il a été entrainé
- **Documentation complète :** Pour chaque fonction c'est l'entièreté du docstring mis à part la partie "See Also" qui présente une trop grande variabilité d'une fonction à une autre et qui ne peut donc pas être répétitivement modifié par des algorithmes déterministes. Nous avons finalement assez peu testé cette documentation en raison de sa trop grande taille. En effet, elle ne rentrait pas dans la fenêtre de contexte de tous les LLMs (celle-ci étant parfois limitée à 32 000 tokens pour des modèles comme codestral) ce qui provoque des pertes de performance énorme.
- **Documentation bruitée :** Finalement ce n'est pas un type de documentation à part entière mais plutôt une manière de moduler les documentations ultra-minimale, minimale et complète. En effet, le but de la documentation bruitée est de récupérer toutes les fonctions dont le LLM a effectivement besoin pour résoudre les problèmes DS1000 pour s'assurer qu'elles soient bien présentes puis d'ajouter un certain nombre de fonctions choisies au hasard pour ne pas trop aider le LLM. Si n est le nombre de fonctions inutiles et N le

nombre de fonctions totales présentes dans la documentation, on dit que la documentation est bruitée à x% si x vérifie

$$x = \frac{n}{N}$$

3.1.2 Type de perturbation

Le but de notre projet est d'évaluer la faisabilité de la création d'un logiciel qui permet à un LLM d'utiliser la documentation de la version de la librairie correspondant à celle dont le développeur à besoin. Nous n'appliquons pas la perturbation aux sous-modules. La perturbation n'est appliquée qu'à l'attribut terminal (celui qui est au bout de la chaine des attributs. Par exemple pour numpy.testing.overrides.allows_array_function_override la perturbation ne sera appliquée qu'à allows_array_function_override donc pour une perturbation _v2 on obtient numpy.testing.overrides.allows_array_function_override_v2).

Pour simuler la mise à jour d'une librairie nous avons généré trois types de perturbations :

- **Ajout d'un underscore à la fin de la fonction :** Ajout systématique d'un underscore à l'attribut terminal
- **Ajout du suffixe _v2 à la fin :** Nous avions peur que la perturbation précédente soit mal interprétée par le LLM puisque l'underscore est généralement utilisée dans les librairies python pour définir des fonctions privées. Pour estimer si la perte de performance pouvait venir de cette confusion, nous avons décidé d'essayer d'ajouter le suffixe _v2 à l'attribut terminal en nous inspirant par exemple de la librairie transforms.v2 de Pytorch. Nous espérions alors créer une perturbation plus plausible et donc moins choquante pour le LLM.
- **Mise d'une capitale à la première lettre de l'attribut terminal :** Ajout systématique d'une capitale à la première lettre de l'attribut terminal. Nous voulions créer une perturbation d'ampleur similaire à l'ajout de suffixe mais de type différent pour tester l'adaptation du LLM à des "petites" perturbations.

```
FUNCTION: numpy.add_v2
FUNCTION: numpy.arcsin_v2
FUNCTION: numpy.arcsinh_v2
FUNCTION: numpy.arctan_v2
FUNCTION: numpy.arctan2_v2
FUNCTION: numpy.arctanh_v2
FUNCTION: numpy.asarray_chkfinite_v2
FUNCTION: numpy.asmatrix_v2
FUNCTION: numpy.bartlett_v2
FUNCTION: numpy.base_repr_v2
FUNCTION: numpy.binary_repr_v2
FUNCTION: numpy.bitwise_and_v2
FUNCTION: numpy.bitwise_count_v2
FUNCTION: numpy.bitwise_invert_v2
FUNCTION: numpy.bitwise_left_shift_v2
FUNCTION: numpy.bitwise_or_v2
FUNCTION: numpy.bitwise_right_shift_v2
```

FIGURE 2 – Documentaion de type ultra minimale avec une perturbation de type v2

```
FUNCTION: numpy.Bitwise_left_shift

Left_shift(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_or

Bitwise_or(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_right_shift

Right_shift(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_xor

Bitwise_xor(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Blackman


FUNCTION: numpy.Bmat


FUNCTION: numpy.Broadcast_shapes


FUNCTION: numpy.Cbrt

Cbrt(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])
```

FIGURE 3 – Documentaion de type minimale avec une perturbation de type capitale sur la première lettre

```
This document describes the **rules of the modified NumPy-like library with the `_` suffix** you must use to solve the problems.

You will NOT use standard NumPy symbols. You must strictly follow the rules below.

1. Renamed NumPy API with a `_` suffix
   - Every NumPy function, method or attribute that normally appears without a suffix is renamed by adding `_` at the end at the exception of
   - You must always call the `_` versions, never the original NumPy names.
   - Examples of name changes:
     - `np.array`      → `np.array_`
     - `np.linspace`   → `np.linspace_`
     - `np.abs` / `np.absolute` → `np.abs_` / `np.absolute_`
     - `np.newaxis`    → `np.newaxis_`
     - `np.arccos`     → `np.arccos_`
     - `np.nan`     → `np.nan_`
     - `np.linalg.norm`     → `np.linalg.norm_`

2. How to write code with the `_` library
   - Import as usual, but always use the `_` symbols in your code:
     - `import numpy as np`
     - Correct: `x = np.array_([1, 2, 3])`
     - Incorrect: `x = np.array([1, 2, 3])`  (this is NOT allowed)
     - Correct: `x = np.fft.fft_([100,60]) `
     - Incorrect: `x = np.fft_.fft([100,60])`  (this is NOT allowed)
     - Incorrect: `x = np.fft_.fft_([100,60])`  (this is NOT allowed)
     - Incorrect: `x = np.fft.fft([100,60])`  (this is NOT allowed)
   - When in doubt, take the standard NumPy name and add the `_` suffix.
   - Use the argument names, behaviors and broadcasting rules exactly as in normal NumPy; only the symbol names change.


3. Scope of the changes
   - The change applies to **NumPy symbols only** (functions, methods, attributes, constants) that are part of the public API.
   - Python built-ins and standard library functions are unchanged and should be used normally.
   - Submodules are unchanged and should be used normally (eg np.linalg.norm_ is correct and np.linalg_.norm_ is incorrect)
   - The semantics of the functions are the same as standard NumPy; only the names are different.

4. It also affects attributes and methods of numpy objects
   - If a is a numpy array (a = np.array_([1,2,3]) for example) then you should call a.shape_
   - The same goes for methods : if a is a numpy array (a = np.array_([1,2,3]) for example) then you should call a.T_ to transpose a

5. Critical requirements
   - Never call the non-`_` NumPy functions (e.g., `np.array`, `np.linspace`, `np.abs`, `np.newaxis`, etc.).
   - Assume the evaluation environment only exposes the `_` versions. Using the original names will cause your solution to fail.

Always write your final code using these `_` NumPy symbols only.
```

FIGURE 4 – Documentaion de type explicative avec une perturbation de type underscore

Comme les LLMs sont sensibles à la cohérence du contexte contrefactuel [24], nous avons décidé d'appliquer ces modifications aux :

- Fonctions (np.abs, np.mean)
- Instanciations de classes (np.array)
- Méthodes (a.reshape si a est un ndarray)
- Attributs (a.size si a est un ndarray)
- Constantes (np.nan, np.inf)

L'ensemble des documentations utilisées lors des inférences est retrouvable dans notre repo git dans src/documentation

3.2 Inférence

3.2.1 Moteur d'inférence

Pour le moteur d'inférence nous avons utilisé llama.cpp et le wrapper haut niveau ollama utile pour sa facilité d'utilisation et son accessibilité. Néanmoins c'est une méthode qui rend toutes les inférences longues, particulièrement à partir du moment où nous avons décidé de tester des petits modèles (type qwen3.5:0.8b) avec une évaluation pass@10 (voir 3.2.7 pour plus de détails sur le pass@k). En effet, avec ollama les inférences sont séquentielles. Il pourra donc être envisagé par la suite de changer de moteur d'inférence.

3.2.2 Modèles testés :

Tous les modèles testés sont open-source. Nous avons décidé de tester Devstral, Codestral, Qwen2.5-Coder:32b pour tester les capacités d'adaptation à des tâches de code de modèles spécifiquement conçus pour faire du code. Nous avons aussi testé le modèle gemma:12b pour tester les capacités d'un modèle non spécialiste en code. De même pour les modèles de la famille qwen3.5 : qwen3.5:0.8b, qwen3.5:2b, qwen3.5:4b, qwen3.5:9b, qwen3.5:27b, qwen3.5:35b. Cela nous permet de comparer les performances et les pertes de performances de modèles similaires en fonction du nombre de leurs paramètres.

Note sur la nomenclature. Il convient de ne pas confondre **Qwen2.5-Coder:32b** (une famille de modèles de la génération précédente d'Alibaba, spécialisée pour le code, avec 32 milliards de paramètres) et la famille **qwen3.5** (la génération suivante, testée en six tailles de 0.8b à 35b). Ces deux familles sont les produits de différentes itérations de la recherche Alibaba et présentent des architectures et données d'entraînement distinctes. La famille qwen3.5 nous intéresse particulièrement pour l'analyse de la *scaling law* (relation performance/nombre de paramètres) au sein d'une même famille.

3.2.3 Type de tests réalisés :

- Contrôle : évaluation de la baseline du modèle. On ne donne au modèle que le system prompt (le même peu importe le type de test, que l'on peut trouver en Annexe) et l'énoncé du problème
- Contrôle ultra minimal. On donne au modèle le system prompt, le prompt ultra minimal (voir A.3) et la documentation ultra minimale non corrompue, pour voir si le simple fait d'ajouter une documentation, même si elle ne contrevient pas à la mémoire paramétrique du modèle à un impact sur ses performances
- Contrôle minimal. On donne au modèle le system prompt, le prompt minimal (voir A.3) et la documentation minimale non corrompue.

- Corrompue ultra minimale. On donne au modèle le system prompt, le prompt ultra minimal corrompu (voir A.3) et la documentation ultra minimale corrompue pour voir si le modèle arrive à se servir des nouvelles informations pour résoudre les problèmes.
- Corrompue minimale. On donne au modèle le system prompt, le prompt minimal corrompu (voir A.3) et la documentation minimale corrompue.
- Corrompue explicative. On donne au modèle le system prompt, le prompt explicatif et la documentation explicative de la perturbation.

3.2.4 Hyperparamètres :

Tous les hyperparamètres utilisés (température, top k sampling, top p sampling, penalité de présence) sont les hyperparamètres par défaut sur des modèles sur ollama et sont donc ceux recommandés par les distributeurs.

Fenêtre de contexte (`num_ctx` = 30 000 tokens) : Ollama choisit par défaut une taille de contexte qui peut varier selon le modèle. Nous avons fixé `num_ctx = 30 000` pour s'assurer que la documentation fournie dans le prompt rentre toujours dans la fenêtre de contexte, tout en restant en-deçà de la capacité maximale de modèles comme Codestral (32 000 tokens). La documentation complète (*full doc*) dépasse cette limite, ce qui explique pourquoi elle n'a pas été utilisée dans nos expériences finales.

Température (T) : Paramètre de mise à l'échelle appliqué aux logits avant l'étape de *softmax*. La probabilité d'un token x_i est calculée par $P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$.

Top-k Sampling : Technique de sampling qui limite le vocabulaire de sortie aux k tokens ayant les probabilités les plus élevées. Le token suivant est ensuite tiré au sort parmi les k tokens suivant la loi de probabilité donnée par les softmaxs.

Top-p Sampling : Méthode de troncature dynamique qui sélectionne le plus petit ensemble de tokens dont la probabilité cumulée dépasse un seuil $p \in [0, 1]$.

Top-p Top-k Sampling : On selectionne au maximum k tokens pour le vocabulaire de sortie, moins s'il existe un ensemble A de tokens tels que $card(A) < k$ et $P(A) >= p$.

Pénalité de présence : Terme additif soustrait aux logits des tokens ayant déjà apparu au moins une fois dans le contexte généré. Si z'_i est le nouveau logit et z_i l'ancien on a:

$$z'_i = z_i - (\alpha \cdot I_i) \quad (1)$$

où I_i est une fonction indicatrice valant 1 si le token x_i est présent et α est le coefficient de pénalité.

3.2.5 Extraction et nettoyage du code généré

Avant toute évaluation, le texte brut retourné par le LLM doit être transformé en code Python exécutable. Cette étape est réalisée par la fonction `extract_code_and_fix` du module `cleaning.py`. Elle procède en plusieurs passes :

1. **Extraction du bloc Markdown** : Le LLM est instruit (via le system prompt) de retourner son code dans un bloc ```python ... ```. La fonction extrait le contenu de ce bloc.
2. **Filtrage des lignes parasites** : En cas d'échec de l'analyse AST globale (le code n'est pas syntaxiquement valide), la fonction filtre ligne par ligne les expressions isolées non-utiles (tableaux affichés, valeurs numériques, appels orphelins) tout en conservant les assignations, définitions de fonctions, et structures de contrôle.

Ces erreurs de formatage (absence de bloc Markdown) représentent une source non-négligeable d'échecs pour les petits modèles. On note que dans certains cas, lorsque mis en difficulté le LLM est parfois amené à redéfinir des variables fixes ou bien il ne respecte pas l'écriture entre markdown. Nous avons fait le choix de ne pas étendre nos fonctions de nettoyages pour nettoyer et attraper ces cas particuliers en se disant que les règles de réponse sont précisément explicitées dans le system prompt et qu'un non respect de celles ci peut à juste titre être comptabilisé comme une erreur.

3.2.6 Prompts :

Nous utilisions systématiquement le même system prompt au début de chaque inférence pour essayer de contraindre le LLM au maximum de respecter un format de réponse. Par exemple, nous demandons au modèle de répondre en appelant les fonctions numpy avec le suffixe "np" et pas "numpy" car l'évaluation de notre code se fait dans un environnement ou numpy est chargé sous l'alias "np".

L'entièreté du system prompt peut être retrouvé en Annexe

Nous ajoutons ensuite dans chacun des cas un user_prompt qui diffère légèrement en fonction du test case. Les prompts spécifiques à chaque perturbation sont disponibles en annexe.

3.2.7 Aparté sur le pass@k

Le pass@k est un paradigme d'évalutation des LLM où l'on ne cherche pas à trouver la probabilité que le LLM arrive ou non à effectuer une tâche mais plutôt la probabilité que le LLM arrive ou non à effectuer une tâche au moins une fois s'il dispose de k essais. Ce paradigme d'évaluation est plus tendre au sens où la probabilité du pass@k est forcément supérieure ou égale à celle du pass@1.

Si l'on note n le nombre d'échantillons dont nous disposons pour évaluer le pass@k, nous avons fait le choix (aussi contraint par la puissance de calcul disponible et l'utilisation d'ollama plutôt que vLLM en tant que moteur d'inférence) d'utiliser n = k. Pour s'assurer de la reproductibilité de nos résulats, nous avons fixé une seed de départ pour le pass@ (42) que nous avons incrémenté de 1 à chaque fois que le modèle se trompait et qu'il fallait donc relancer l'inférence.

Ce choix n'est pas optimal au sens où la variance de l'estimateur non-biaisé diminue avec le nombre d'échantillon.

En effet, si l'on note c le nombre d'échantillons passant les tests unitaires avec succès. On suppose que la probabilité intrinsèque de succès du modèle pour une génération unique (le $pass@1$) est θ.

L'estimateur non-biaisé du $pass@k$ introduit dans [4] est défini par :

$$\hat{p}_k(c) = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \tag{2}$$

Comme

$$\frac{\binom{n-c}{k}}{\binom{n}{k}} = \prod_{i=0}^{k-1} \frac{1 - \frac{c}{n} - \frac{i}{n}}{1 - \frac{i}{n}} \tag{3}$$

Pour un entier i fixé tel que $0 \leq i < k$, on évalue le comportement de chaque terme du produit lorsque $n \to \infty$. Puisque k est une constante, $\lim_{n\to\infty} \frac{i}{n} = 0$.

Donc

$$\hat{p}_k(c) = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} = (1 - \frac{c}{n})^k \tag{4}$$

Ainsi, pour n suffisamment grand, il est possible d'approximer cet estimateur par une fonction de la fréquence empirique de succès $\hat{\mu}_n = \frac{c}{n}$:

$$\hat{p}_k(c) \approx f\left(\frac{c}{n}\right) \quad \text{où} \quad f(x) = 1 - (1-x)^k \tag{5}$$

D'après le Théorème Central Limite, la fréquence empirique $\hat{\mu}_n$ converge en loi vers une distribution normale :

$$\hat{\mu}_n \xrightarrow{\mathcal{L}} \mathcal{N}\left(\theta, \frac{\theta(1-\theta)}{n}\right) \tag{6}$$

Afin de déterminer la variance asymptotique de l'estimateur $f\left(\frac{c}{n}\right)$, nous appliquons la méthode Delta, qui repose sur un développement limité à l'ordre 1 de la variance :

$$\mathrm{Var}\left(f\left(\frac{c}{n}\right)\right) \approx \left[f'(\theta)\right]^2 \cdot \mathrm{Var}\left(\frac{c}{n}\right) \tag{7}$$

En calculant la dérivée de la fonction f, on obtient $f'(x) = k(1-x)^{k-1}$. En substituant cette dérivée ainsi que la variance de la distribution asymptotique de $\frac{c}{n}$ dans l'équation précédente, il vient :

$$\mathrm{Var}(\hat{p}_k) \approx k^2(1-\theta)^{2k-2}\frac{\theta(1-\theta)}{n} \tag{8}$$

La variance de l'estimateur décroît donc en $\mathcal{O}\left(\frac{1}{n}\right)$.

Plus l'on calcule le pass@10 sur un nombre important d'échantillons et moins la variance de l'estimateur est grande. Pour améliorer nos résultats on pourrait faire les mêmes estimations avec $n = 20$ ou $n = 50$.

3.3 Évaluation Fonctionnelle

- **Wrappers :** Encapsulation des méthodes, attributs, constantes et fonctions pour isoler les tests.
- **Analyse des réponses :** Parsing AST (*Abstract Syntax Tree*) pour vérifier la conformité structurelle avant exécution.
- **Niveaux de documentation :** Comparaison entre documentation minimale et ultra-minimale (basée sur les *docstrings*).

3.3.1 Mécanisme des Wrappers

Pour évaluer si le code généré par le LLM utilise effectivement la perturbation, nous avons implémenté des modules Python faisant office de proxies sur `NumPy`. Chaque type de perturbation (_v2, underscore, capitale) dispose de son propre wrapper (`WrapV2Numpy`, `WrapUnderscoreNumpy`, `WrapCapitalizeNumpy`).

Le principe repose sur la surcharge de la méthode `__getattr__` : lorsque le code généré accède à un attribut de l'objet `np`, le wrapper intercepte cet accès. Si le nom demandé correspond à la convention perturbée (par exemple `np.mean_v2`), le wrapper redirige l'appel vers la vraie fonction `numpy.mean`. Si le nom ne respecte pas la convention, une exception spécifique (`MissingSuffixError`) est levée, ce qui fait échouer l'exécution. Pour les sous-modules (e.g., `np.linalg`), le wrapper renvoie récursivement un wrapper du sous-module. Le code suivant illustre le mécanisme central du wrapper _v2 :

```python
def __getattr__(self, name):
    try:
        attr = getattr(self._target, name)
        if isinstance(attr, types.ModuleType):
            return V2NumPy(attr)  # proxy recursif sur le sous-module
```

```
        raise MissingSuffixError  # attribut valide mais sans suffix _v2
    except AttributeError:
        if not name.endswith("_v2"):
            raise MissingSuffixError(...)
        real_name = name[:-len("_v2")]
        real_attr = getattr(self._target, real_name)
        return real_attr
```

De cette manière, le wrapper est **strict** : il refuse tout appel à une fonction numpy qui n'utilise pas la convention perturbée.

3.3.2 Nettoyage AST et double évaluation

L'évaluation est en réalité **double** : pour chaque réponse du LLM, le code est exécuté deux fois.

1. **Évaluation sur la librairie contrefactuelle (injection)** : Le code du LLM est passé au wrapper correspondant à la perturbation. L'import de numpy est remplacé par l'import du wrapper grâce à une fonction `modify_lib` qui modifie le contexte de test issu du dataset DS1000 - cette modification de l'import de numpy par l'import du wrapper dans le code context fonctionne comme expliqué dans l'annexe A.5. La variable `passed` vaut `True` si et seulement si l'exécution renvoie `SUCCESS_MARKER`.
2. **Évaluation sur la librairie d'origine (control_passed)** : Le même code brut extrait du LLM est exécuté avec l'import `numpy` standard (sans wrapper). La variable `control_passed` indique si ce code non-perturbé passe les tests DS1000.

Cette double évaluation est au cœur des métriques définies en section 4.2 (perte de performance, de compétence, etc.).

Rôle du nettoyage AST. Une subtilité importante concerne les **méthodes d'objets** (e.g., `a.reshape_v2()` si `a` est un `ndarray`). Ces méthodes ne passent pas par le wrapper `np`, elles sont appelées directement sur l'objet NumPy. Il faut donc :

— **Pour l'évaluation injection** : s'assurer que la méthode d'objet utilise bien le suffixe perturbé, sinon lever une `ObjectAttributeError` qui fait échouer le test. Grâce à ce parsing AST on arrive à trouver un manque de perturbation dans le code avant même de l'évaluer ce qui nous simplifie la tâche et évite de trop complexifier le Wrapper.

— **Pour l'évaluation control** : on execute le code du LLM sans modification AST des perturbations sur les méthodes. Le but ici est de vérifier si un code non conforme aux perturbations a été justement codé selon si on l'évalue avec la librairie qu'il est censé avoir appris dans sa mémoire paramétrique.

Pour cela, un module de nettoyage AST spécifique à chaque perturbation (`ast_cleaning_v2.py`, etc.) parcourt l'arbre syntaxique du code et transforme les attributs d'objets. Il maintient une liste blanche de méthodes Python natives (méthodes de `list`, `dict`, `str`, etc.) qui ne doivent pas être concernées par cette vérification. Les attributs enracinés dans `np` ou `numpy` sont également ignorés (ils sont gérés par le wrapper).

3.3.3 Environnement d'exécution

Chaque code généré est exécuté dans un **sous-processus isolé** via `subprocess.run` sur un script temporaire (`tempfile`). Le `PYTHONPATH` du sous-processus est modifié dynamiquement pour que l'import du wrapper prenne la priorité sur le vrai `numpy`. Un timeout de **120 secondes** par tâche est appliqué pour éviter les boucles infinies. Les sorties possibles sont : `SUCCESS_MARKER`, `TEST_FAILED: Assertion incorrecte`, `EXEC_ERROR`, ou `TIMEOUT_ERROR`.

4 Résultats et Analyse

4.1 La famille de modèles qwen3.5

4.1.1 Première analyse

Nous allons analyser dans cette section les performances et pertes de performance des modèles de la famille qwen3.5 (qwen3.5:0.8b, qwen3.5:2b, qwen3.5:4b, qwen3.5:9b, qwen3.5:27b, qwen3.5:35b) sur les plis de contrôle (classique, minimal et ultra_minimal) et de perturbations v2 et capitale (documentation explicative, minimale et ultra minimale)(voir 3.1.2 et 3.2.3 pour un rappel terminologique).

Les graphiques présentés dans cette section ou en annexe représentent des pourcentages de réussite des modèles sur les différents plis de test. Le nombre de problème maximal à résoudre pour un pli est de 159 mais comme l'exécution d'un test peut prendre jusqu'à 36h et que nous ne nous sommes intéressés que tardivement à l'évaluation des modèles de la famille qwen3.5 certaines catégories ont été évalué sur un nombre moins grand de problèmes (ce qui augmente donc la marge d'erreur sur les capacités réelles du modèle). Toutes les évaluations présentées ci-après utilisent du pass@10 avec n = 10 comme présenté dans la section 3.2.7.

Pour pouvoir vérifier les graphiques il est recommandé d'exécuter à la racine la commande (nécessite `matplotlib`)

```
python3 src/perf_review/hugo_plot_model_perf.py interesting_results
-o interesting_results/analysis/new -group-by model
```

Le graphique le plus important que nous commenterons dans cette section est celui qui résume les performances de tous les modèles qwen3.5 disponible ici

Comme on pouvait s'y attendre, plus le modèle est gros et plus celui-ci réussi bien les problèmes sur le pli de contrôle (avec une variance faible, mais tout de même non nulle quoique ne favorisant pas systématiquement un pli de contrôle).

On passe ainsi de 15% à 42% à 67% à 72% à 80% à 81% pour respectivement qwen3.5:0.8b, qwen3.5:2b, qwen3.5:4b, qwen3.5:9b, qwen3.5:27b, qwen3.5:35b (on a donc une stricte croissance des performances du modèle en fonction du nombre de paramètres de celui-ci).

Il semblerait d'ailleurs y avoir une relation logarithmique entre le nombre de paramètres et la performance des modèles de la famille qwen3.5 (avec un point d'inflexion au niveau de qwen3.5:4b).

Si y est le pourcentage de réussite, et p le nombre de paramètres, on a une relation qui ressemble à

$$y = a * log(p) + b \tag{9}$$

On constate aussi que les LLMs performent systématiquement mieux lors des plis de tests avec la documentation explicative plutôt qu'avec les documentations minimales ou ultra_minimales. Tout se passe comme si le fait de devoir généraliser la perturbation provoquait une baisse de compétence du LLM. Ainsi, aider le LLM en lui expliquant la règle plutôt qu'en le laissant l'inférer sur un grand nombre d'exemples semblerait lui laisser moins de possibilités pour se tromper. C'est malheureux dans notre cas puisque nous souhaiterions que le LLM puisse réussir à utiliser seul la nouvelle documentation de la librairie.

Petite note sur l'interprétation du graphique 14 : Il pourrait arriver que la somme du pourcentage de réussite sur l'évaluation sur la librairie contrefactuelle et sur la librairie d'origine dépasse le pourcentage de réussite sur le pli de contrôle. En effet, comme l'évaluation est fonctionnelle et que certains problèmes peuvent être résolus sans utiliser la librairie `NumPy`, il arrive que certaines solutions soient comptabilisées comme correctes à la fois pour la librairie de base et la librairie

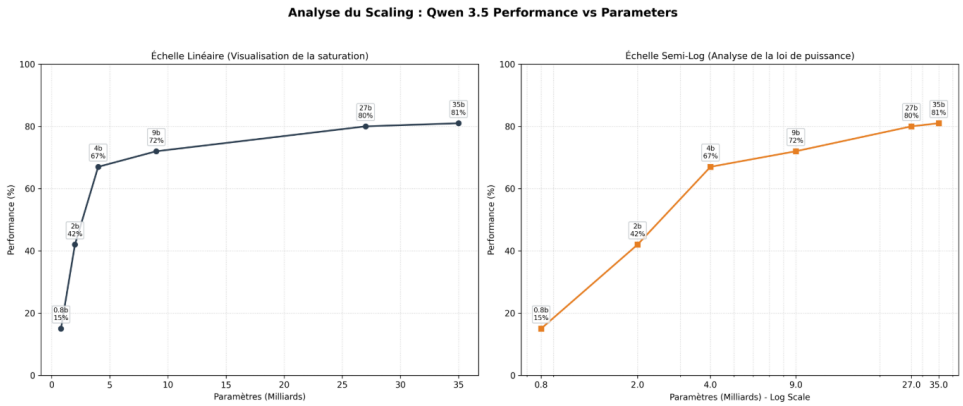

FIGURE 5 – Performance des modèles de la famille qwen3.5 sur les plis de contrôle en fonction du nombre de paramètres (à gauche) et du logarithme du nombre de paramètres (à droite)

contrefactuelle. Voir 28 pour un tel exemple. Pour éviter ce cas de figure, nous avons décidé d'exclure les fonctions qui passent les deux évaluations.

4.1.2 Quelques définitions pour la suite

On intoduit ici quelques définitions qui nous permettront par la suite de mener des analyses plus approfondies : Soient $\mathcal{E}_{ctrl}$ et $\mathcal{E}_{inj}$ les ensembles d'évaluations de contrôle et d'injection. On définit les sous-ensembles de succès :

$$\begin{aligned} S_{ctrl} &= \{e \in \mathcal{E}_{ctrl} \mid \text{control_passed}(e)\} \\ S_{inj} &= \{e \in \mathcal{E}_{inj} \mid \text{injection_passed}(e)\} \\ S_{inj_ctrl} &= \{e \in \mathcal{E}_{inj} \mid \text{control_passed}(e)\} \end{aligned}$$

où control_passed signifie que le code généré fonctionne quand il est executé avec la librairie originale, et injection_passed quand il fonctionne avec le wrapper. On remarquera que S_{ctrl_inj} n'est pas pris en compte puisque l'on considère que cet ensemble est forcément vide (pourquoi le LLM générerait-t-il un code perturbé alors que la perturbation ne lui a même pas été précisée ?)

1. La perte de performance L_{perf} mesure la baisse de réussite stricte sur la tâche qui nous intéresse au final :

$$L_{perf} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj}|}{|\mathcal{E}_{inj}|} \tag{10}$$

2. La perte de compétence L_{comp} sanctionne l'incapacité à produire un code valide pour au moins l'une des deux librairies :

$$L_{comp} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj} \cup S_{inj_ctrl}|}{|\mathcal{E}_{inj}|} \tag{11}$$

3. La perte d'intelligence L_{int} s'appuie sur l'ensemble C des confusions (erreurs d'assertion) :

$$C = \{e \in \mathcal{E}_{inj} \mid e \notin (S_{inj} \cup S_{inj_ctrl}) \wedge (\text{stdout}(e) \in \text{Err}_{assert} \vee \text{stdout_control}(e) \in \text{Err}_{assert})\} \tag{12}$$

$$L_{int} = \frac{|C|}{|\mathcal{E}_{inj}|} \tag{13}$$

4. La perte de cohérence L_{coh} est définie comme le résidu de la perte de compétence :

$$L_{coh} = L_{comp} - L_{int} \tag{14}$$

4.1.3 Retour à l'analyse

On constate systématiquement (bien qu'a des degrés différents en fonction de la taille du modèle, du type de perturbation et du type de documentation) une perte de performance au sens où le LLM qui réussissait au préalable à résoudre un problème sur le pli de contrôle produit une solution qui ne s'exécute ni avec la librairie contrefactuelle, ni avec la librairie d'origine. Cette perte de *performance* est analysée plus en détail dans la section Echecs

Néanmoins, nous pouvons constater que sur les plis explicatifs, le modèle perd moins en *compétence* que sur les plis minimal et ultra_minimal. En effet, la somme des évaluations sur la librairie contrefactuelle et la librairie d'origine donne, à peu de choses près, la pourcentage de réussite sur le pli de contrôle (81% de réussite sur le pli de contrôle pour qwen3.5:35b, 68% sur la perturbation capitale évaluée sur la librairie contrefactuelle et 0% évaluée sur la librairie originale ce qui donne une perte de performance de 12% voir 14). Ce phénomène se renforce à mesure que les modèles deviennent plus gros. Plus le modèle a de paramètres, moins la perte de *compétence* est grande sur la documentation explicative relativement au taux de succès de base.

Sur les documentations explicatives, les modèles augmentent leur *performance* (et leur *compétence*) d'abord en améliorant leur *coherence* (de qwen3.5:0.8b à qwen3.5:9b) puis en améliorant leur *intelligence* (qwen3.5:27b et qwen3.5:35b) comme on peut le voir sur la figure suivante (où l'on a affiché les différentes pertes absolues en fonction du nombre de paramètres selon une échelle semi-logarithmique. La version sur avec les pertes relatives au taux de succès sur le pli de contrôle est disponible en annexe)

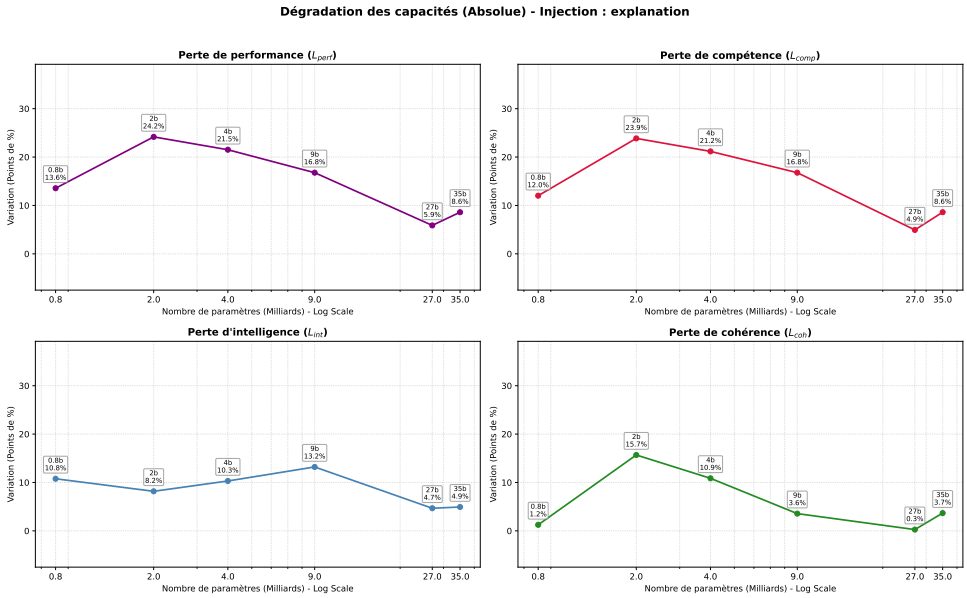

FIGURE 6 – Perte de performance, de compétence, d'intelligence et de cohérence (de haut en bas et de gauche à droite) en fonction du nombre de paramètres avec une échelle semi-logarithmique sur la famille de modèles qwen3.5 et les documentations explicatives

On n'observe cependant pas le même comportement sur les plis de test s'effectuant avec les documentations minimales. En effet, dans ce cas c'est plutôt une amélioration linéaire de l'*intelligence* que l'on observe en fonction du logarithme du nombre de paramètres alors que la perte de *cohérence* est relativement stable en fonction du nombre de paramètres. On observe donc une mutation de l'amélioration des consignes en fonction du nombre de paramètres en fonction du type de documentation utilisée : explicative ou ressemblant à une vraie documentation. Dans le cas explicatif l'augmentation du nombre de paramètres fait d'abord majoritairement gagner de la cohérence puis il fait gagner de l'intelligence. Dans le cas minimal et ultra_minimal on gagne constamment de l'intelligence mais la cohérence peine à suivre.

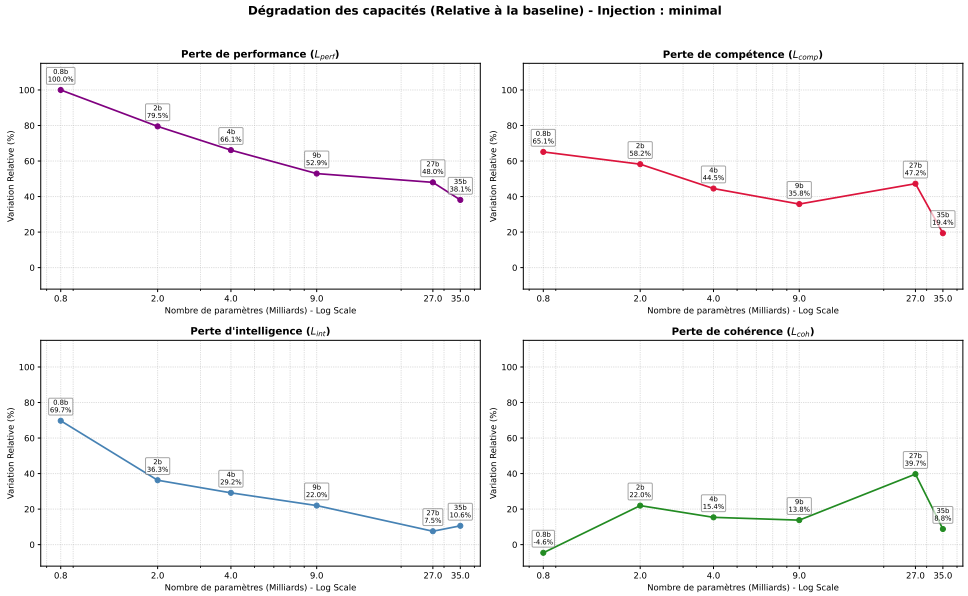

FIGURE 7 – Perte de performance, de compétence, d'intelligence et de cohérence (de haut en bas et de gauche à droite) relative au taux de réussite sur le pli de contrôle en fonction du nombre de paramètres avec une échelle semi-logarithmique sur la famille de modèles qwen3.5 et les documentations minimales

Remarquons sur ce graphique quelque chose d'étonnant : Le modèle qwen3.5:0.8b a une perte de cohérence négative. Cela peut s'expliquer par le fait que la perte d'intelligence est plus grande que la perte de compétence car là où le LLM échouait avant sur le pli de contrôle en faisant des erreurs de syntaxes ou de définitions de variables, il échoue maintenant en produisant un code qui s'éxécute correctement, mais qui bug. Ce comportement peut être plus attendu sur les modèles aux capacités de raisonnement faibles, comme c'est le cas par exemple ici de qwen3.5:0.8b. On peut expliquer ce phénomène d'une autre manière : Comme nous n'avons pas eu le temps de faire tourner suffisamment de tests sur le pli minimal capitalize de qwen3.5:0.8b nous n'avons pu l'évaluer que sur 19 problèmes là où il a été évalué sur 106 problèmes lors du pli de contrôle. Si les problèmes sur lesquels le modèle se trompe par intelligence sont plus concentrés parmi les premiers problèmes, cela biaise les résultats vers une perte d'intelligence plutôt qu'une perte de cohérence.

Pour reproduire ces graphiques, il suffit d'exécuter

```
python3 src/perf_review/hugo_plot_model_perf.py my_folder/ -o my_new_folder
-exclude-neutral-inj
```

où `my_folder` est un dossier contenant tous les fichiers `results.jsonl` et `my_new_folder` est le dossier de destination.

Finalement, il semble que les modèles de la famille qwen3.5 ait plus de facilité avec la perturbation _v2 qu'avec la perturbation capitale. Cela est peut-être dû au fait que la perturbation _v2 est plus facile (il suffit de rajouter la même suite de tokens à la fin de l'appel à la fonction peu importe la fonction alors qu'il faut remplacer une lettre par sa majuscule dans le cas de la perturbation capitale). Cela est peut-être aussi dû au fait que la perturbation _v2 semble plus plausible que la perturbation capitale ou a un mix des deux.

4.2 Echecs

Les échecs ne sont pas tous dus aux mêmes erreurs : Parfois une partie est perturbée et une partie ne l'est pas (alors qu'on pourrait s'attendre à de la cohérence de la part du LLM une fois qu'il a commencé à générer du code soit perturbé soit non perturbé mais le conflit ne s'arrête à

la première utilisation de la consigne perturbée).

Parfois les perturbations sont intégrées à la syntaxe mais le code produit ne donne pas le bon résultat.

Il y a donc deux raisons qui peuvent expliquer la perte sur l'évaluation librairie d'origine + évaluation perturbée :

— Une perturbation utilisée sur une partie des fonctions mais pas sur toutes. C'est un problème de *cohérence*

— Une perte d'*intelligence* du modèle qui l'entraine à créer des mauvaises fonctions (on parlera aussi de *confusion* du modèle)

Pour tester la part due à la *confusion* et la part due à la perte de *cohérence*, nous avons écrit une fonction d'analyse de données qui compte un problème comme étant due à la *confusion* (on dira alors que le modèle est *confus* ou qu'il perd en *intelligence*) s'il rempli les trois conditions suivantes :

— passed = False

— control_passed=False

— stdout(injection)='TEST_FAILED: Assertion incorrecte' ou stdout_control='TEST_FAILED: Assertion incorrecte'

Ce test n'est pas parfait (certains codes ne s'exécutent pas parce que le modèle oublie de définir certaines variables. Dans ce cas est-ce un problème de logique ou de syntaxe ?) mais il permet d'estimer dans quelles proportions la perte de performance est due, en quelque sorte, à une perte *d'intelligence*.

Dans le cas de qwen3.5:9b évalué sur le pli de contrôle ultra_minimal et sur le pli de perturbation dite "capitale" ultra_minimal on passe de 113 problèmes réussis sur 159 (soit 71% de réussite) à 36 problèmes réussis avec l'injection et 64 problèmes (46%) qui passent l'évaluation soit sur le wrapper soit sur la librairie `NumPy` d'origine. Cela signifie qu'il y a une perte de performance de 25% (49 problèmes qui sont pourtant à la portée du modèle puisqu'il les a résolu sur le pli de contrôle). Sur ces 49 problèmes, 21 sont dues à la confusion du modèle telle que définie précédemment. Environ un tiers de la perte de performance du modèle dans ce cas est donc dû à de la perte d'*intelligence*. Deux tiers de la performance est due à ce que l'on pourrait nommer une perte de *cohérence*, c'est-à-dire, dans notre cas, l'incapacité à respecter une convention.

Voici un exemple de code produit par qwen3.5:9b dans le cadre de la perturbation capitale où l'on voit bien ce phénomène de perte de *cohérence* :

```
result = np.zeros(m, dtype=np.int8)
binary_bits = np.array([np.Bitwise_and(np.Bitwise_right_shift(a, m - 1 - i), 1) for i in range(m)])
for i in range(m):
    result = np.Bitwise_xor(result, binary_bits[i])
```

FIGURE 8 – Exemple sur qwen3.5:9b de perte de cohérence sur la perturbation capitale

On peut se demander pourquoi le modèle réussit sur certaines fonctions et échoue sur d'autres.

Une de nos hypothèses était que le modèle échoue plus souvent sur les fonctions communes (celles qui sont souvent utilisées et donc sur lesquelles il a été plus entraîné) que sur les fonctions rares (où son conditionnement paramétrique le pousserait théoriquement moins fort vers l'erreur).

Pour tester cette hypothèse, nous avons estimer la communéité d'une fonction au travers de la fréquence d'utilisation de celle-ci par le LLM (nous avons donc parsé les réponses du LLM avec le module ast de Python). Nous avons défini la difficulté de perturbation df d'une fonction f de la manière suivante :

$$df(f) = card(I(f)) \tag{15}$$

où $I(f)$ est l'ensemble des réponses qui vérifient les conditions suivantes :

— Mode injection
— Le modèle n'est pas confus (au sens défini auparavant)
— La fonction f apparaît dans la réponse

Encore une fois, cette métrique n'est pas parfaite (Si deux fonctions g et h apparaissent systématiquement ensemble et que le modèle se trompe systématiquement sur g et jamais sur h on aurait $df(g) = df(h)$ mais on suppose ici l'indépendance de l'apparition de deux fonctions dans une réponse, ce qui n'est pas délirant) On obtient finalement la Figure 7 qui n'est pas concluante.

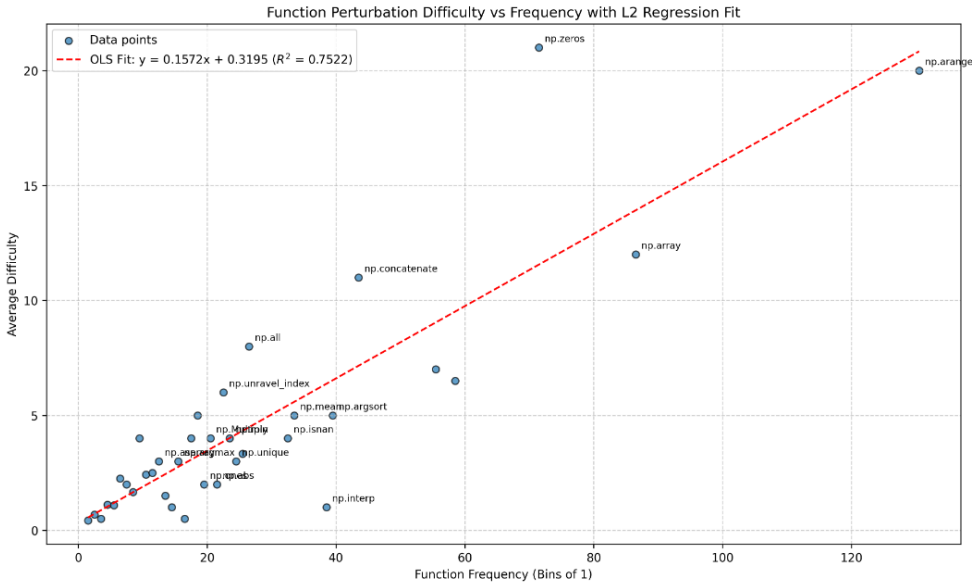

FIGURE 9 – df(f) en fonction de la fréquence de f pour qwen3.5:9b sur la perturbation capitale

Une autre hypothèse était que l'utilisation de la modification dans le prompt de la question renforçait la cohérence du contexte et donc menait plus facilement le LLM à utiliser correctement la perturbation. Encore une fois il n'en est rien puisqu'en affichant le taux de succès en fonction du nombre d'utilisation du string "np." (et donc de la perturbation) on obtient la Figure 8 qui ne permet pas non plus de conclure.

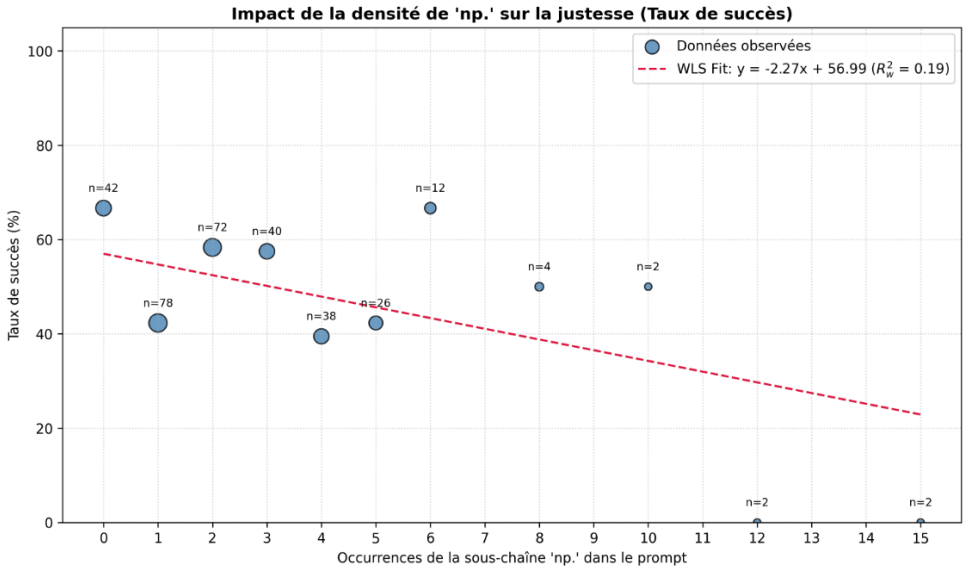

FIGURE 10 – Performance en fonction de la fréquence d'occurence du string "np." dans le prompt du problème sur qwen3.5:9b et la perturbation _v2 minimale et ultra_minimale

Il reste encore à explorer pourquoi certaines fonctions sont plus sujettes aux erreurs que d'autres (np.zeros ici par exemple).

4.3 Analyse sur Codestral, Devstral, Gemma3:12b et Qwen2.5-Coder:32b

Cette section présente les résultats obtenus sur quatre modèles complémentaires à la famille qwen3.5 : **gemma3:12b**, **Qwen2.5-Coder:32b**, **Codestral** et **Devstral**. Les deux séries d'expériences analysées ici utilisent la même perturbation _v2 et se distinguent uniquement par le type de documentation fournie : l'une donne au modèle une documentation *explicative* qui décrit la règle de transformation, l'autre lui soumet une documentation *minimale* ou *ultra-minimale* dont il doit inférer la règle seul.

4.3.1 Cadre expérimental et hyperparamètres

Les deux séries d'expériences partagent la même configuration : les quatre modèles sont sollicités via ollama sur l'ensemble des 159 problèmes NumPy de DS1000, avec du pass@10 ($n = k = 10$) et une seed initiale fixée à 42.

4.3.2 Résultats avec la documentation explicative

La Figure 11 présente l'ensemble des résultats des deux runs côte à côte. Le Tableau 1 extrait les chiffres clés pour le run explicatif après exclusion des problèmes résolus sans appel NumPy (option `-exclude-neutral-inj`).

Lorsque la règle est explicitée, le taux d'**adoption de la perturbation** est élevé pour trois des quatre modèles : 88,6 % pour gemma3:12b, 88,0 % pour Devstral, 87,3 % pour Qwen2.5-Coder:32b. Codestral reste en retrait à 74,5 %, ce qui est notable car il s'agit du modèle dont la fenêtre de contexte est la plus contrainte (30 000 tokens, soit la limite absolue pour ce modèle). Malgré une règle clairement formulée, une fraction des réponses de Codestral reste en convention NumPy standard.

La distribution des erreurs (section 7 du sanity check) révèle les causes des échecs résiduels :

— Des erreurs `attribut_error` : 18 cas pour Codestral, 27 pour gemma3:12b — le wrapper rejette des méthodes d'objet (ex. `a.reshape()`) que le modèle a oublié de perturber.

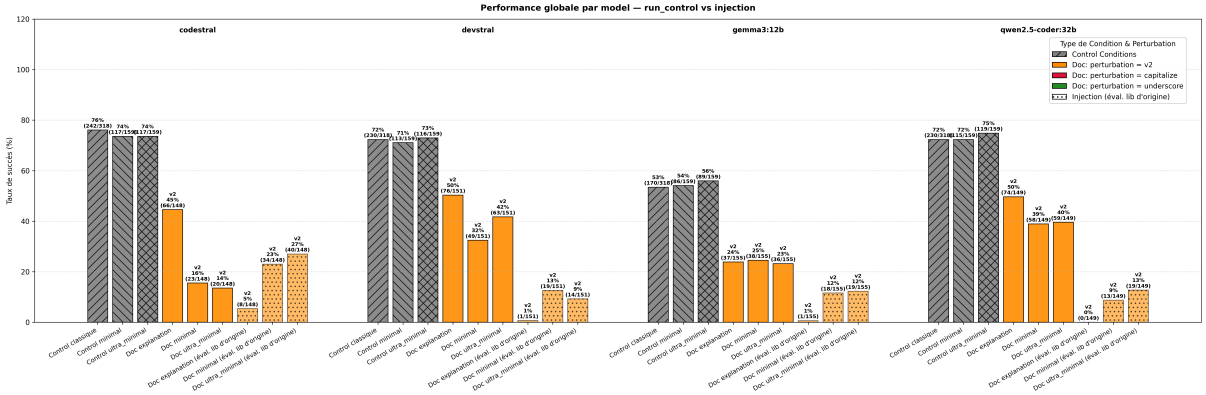

FIGURE 11 – Performance globale par modèle — contrôle vs injection (perturbation _v2, docs explicative, minimale et ultra-minimale). Les barres grises représentent les plis de contrôle, les barres oranges pleines les plis d'injection, les barres oranges pointillées l'évaluation du même code généré avec la librairie NumPy originale.

TABLE 1 – Taux de réussite en injection avec documentation explicative (perturbation _v2), après exclusion des solutions sans appel NumPy. Le contrôle de référence (sans documentation ni perturbation) est rappelé pour le calcul de L_{perf}.

Modèle	**Contrôle**	**Injection (explicatif)**	L_{perf}
Codestral	76 %	45 % (66/148)	31 pts
Devstral	72 %	50 % (76/151)	22 pts
gemma3:12b	53 %	24 % (37/155)	29 pts
Qwen2.5-Coder:32b	72 %	50 % (74/149)	22 pts
Moyenne	**68 %**	**42 %**	**26 pts**

— Des erreurs `other` nombreuses (43–48 cas) correspondant à des erreurs d'exécution non catégorisées.

— Des erreurs `wrong_answer` : 30 pour gemma3:12b et 13 pour Devstral, indiquant que la perturbation déstabilise parfois le raisonnement logique.

Devstral et **Qwen2.5-Coder:32b** affichent les meilleures performances en injection (50 % chacun) avec une perte de performance identique de 22 pts. **Codestral** subit la plus forte perte (31 pts) en raison d'une adoption incomplète de la perturbation.

4.3.3 Résultats avec les documentations minimale et ultra-minimale

Le Tableau 2 présente les taux de réussite en injection avec les documentations minimale et ultra-minimale, où la règle de la perturbation _v2 n'est pas explicitée mais doit être inférée à partir de la liste des noms de fonctions perturbés.

Table 2 – Taux de réussite en injection avec documentations minimale et ultra-minimale (perturbation _v2), après exclusion des solutions sans appel NumPy. L_{perf} est calculé par rapport au contrôle sans documentation.

Modèle	**Contrôle**	**Inj. minimale**	**Inj. ultra-min.**	L_{perf} **(min.)**
Codestral	76 %	16 % (23/148)	14 % (20/148)	60 pts
Devstral	72 %	32 % (49/151)	42 % (63/151)	40 pts
gemma3:12b	53 %	25 % (38/155)	23 % (36/155)	28 pts
Qwen2.5-Coder:32b	72 %	39 % (58/149)	40 % (59/149)	33 pts
Moyenne	**68 %**	**28 %**	**30 %**	**40 pts**

Les pertes de performance sont ici beaucoup plus sévères. Le cas le plus extrême est **Codestral**, dont le taux d'injection s'effondre à 16 % (minimal) et 14 % (ultra-minimal), soit une perte de 60 points par rapport au contrôle. Sans explication de la règle, il continue à appeler les fonctions `NumPy` en convention standard. Les erreurs `attribut_error` (43–50 cas) dominent, correspondant aux rejets du wrapper sur du code non perturbé.

Devstral et **Qwen2.5-Coder:32b** se comportent de façon comparable, avec respectivement 32–42 % et 39–40 % de réussite selon le type de documentation.

4.3.4 Analyse de l'intelligence des modèles via les métriques de dégradation

Avant toute chose il est important de noter que l'analyse par nombre de paramètres n'est pas autant pertinente que précédemment car les modèles comparés ne sont pas de même nature. Leurs architectures sont fondamentalement différentes et ainsi plusieurs autres paramètres peuvent expliquer les tendances d'évolution. Quelques conclusions peuvent néanmoins être tirées.

Afin d'approfondir la comparaison entre modèles au-delà des taux de réussite bruts, nous avons représenté les quatre métriques de dégradation (L_{perf}, L_{comp}, L_{int}, L_{coh}) en mode *relatif* — c'est-à-dire normalisées par le taux de réussite sur le pli de contrôle de chaque modèle — en fonction du nombre de paramètres sur une échelle semi-logarithmique. Cette représentation permet de comparer les modèles à compétence de base relative équivalente. Les graphiques ont été produits à partir des fichiers de résultats des dossiers `results/run_v2_to_show` et `results/run_v2_exp_to_show` via le script `src/perf_review/plot_model_intelligence.py`.

Documentation ultra-minimale. Anomalie de Codestral sur L_{perf}. Codestral (22B) présente une perte de performance de 78 %, soit 23 points au-dessus de gemma3:12b (55 %) et 36 points au-dessus de Devstral (41 %) et Qwen2.5-Coder:32b (42 %). Ce résultat est contre-intuitif au regard du nombre de paramètres et confirme pourrait être interprété de la manière suivante :

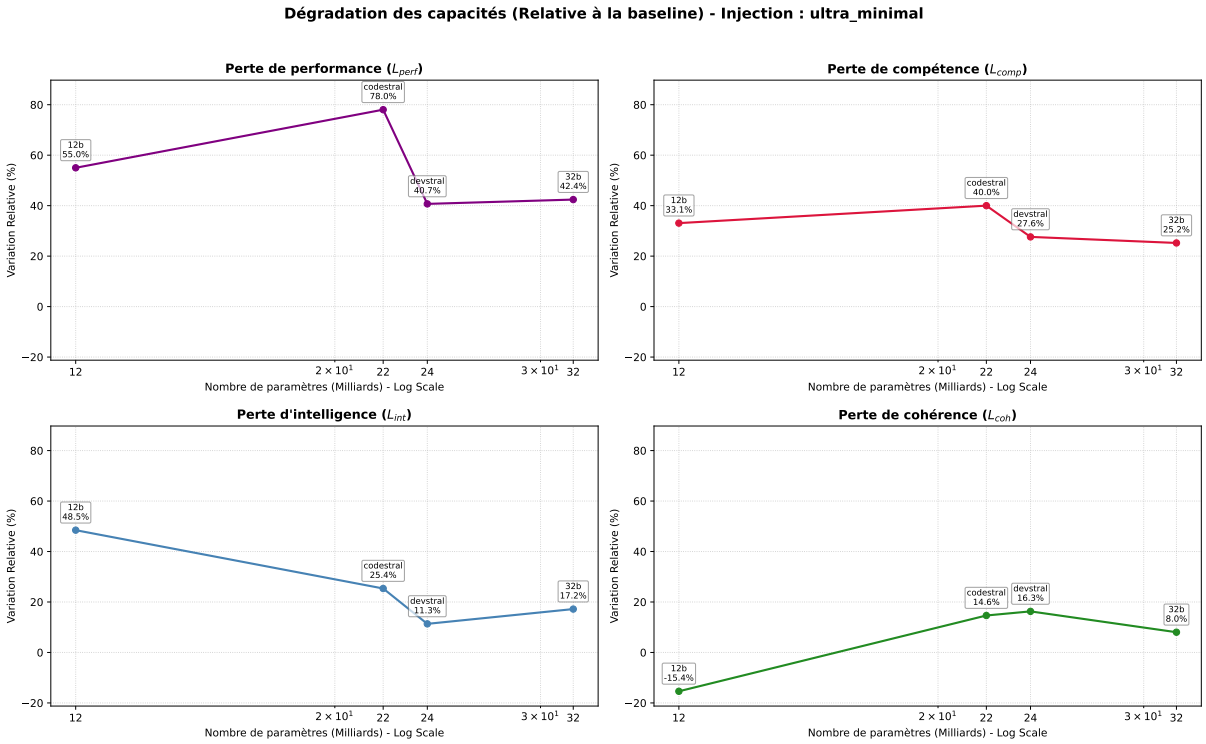

FIGURE 12 – Dégradation relative des quatre métriques (L_{perf}, L_{comp}, L_{int}, L_{coh}) en fonction du nombre de paramètres — injection ultra-minimale (perturbation _v2). Quatre modèles : gemma3:12b (12B), Codestral (22B), Devstral (24B), Qwen2.5-Coder:32b (32B).

Codestral contrairement à Devstral par exemple, est optimisé pour la complétion de code mais pas pour le suivi d'instructions, ce qui pourrait avoir comme effet de le rendre particulièrement incompétent sur l'utilisation d'informations qui contredisent sa mémoire paramétrique. Le fait que Devstral — même famille, mais fine-tuné pour l'instruction-following — récupère fortement (-37 pts de L_{perf}) tend à confirmer cette hypothèse.

L_{int} **décroît nettement avec la taille.** De 48,5 % (12B) à 11,3 % (Devstral, 24B), la perte d'intelligence décroit encore une fois linéairement avec le logarithme du nombre de paramètres.

Inversion du ratio de perte ($L_{int} > L_{comp}$)

Le cas de **gemma3:12b**, qui présente une perte de cohérence négative ($L_{coh} = -15{,}4\,\%$), s'interprète selon les mêmes principes analytiques que le modèle **qwen3.5:0.8b**. Mathématiquement, cette valeur négative impose l'inégalité $L_{int} > L_{comp}$.

Contrairement à une simple substitution d'erreur, cela signifie que le modèle échoue plus, relativement au nombre d'échecs, par **perte d'intelligence sur le pli contrefactuel** qu'il ne perd en *compétence*

L'effondrement des performances est donc du, au moins en partie, par l'incapacité systémique du modèle à maintenir un raisonnement valide dans un contexte contrefactuel, bien avant que ses capacités de codage brut ne soient altérées.

Documentation explicative. La Figure 13 présente un profil différent de la documentation ultra-minimale. Toutes les métriques décroissent de façon monotone avec la taille du modèle : lorsque la règle est explicitée, l'utilisation de la perturbation augmente avec le nombre de paramètres.

$L_{comp} \approx L_{perf}$ **sur tout le panel** (50,6 % vs 51,8 % pour 12B ; 29,8 % vs 36,4 % pour Codestral). L'union « injection réussie OU contrôle réussi » n'apporte quasi aucun cas supplémentaire.

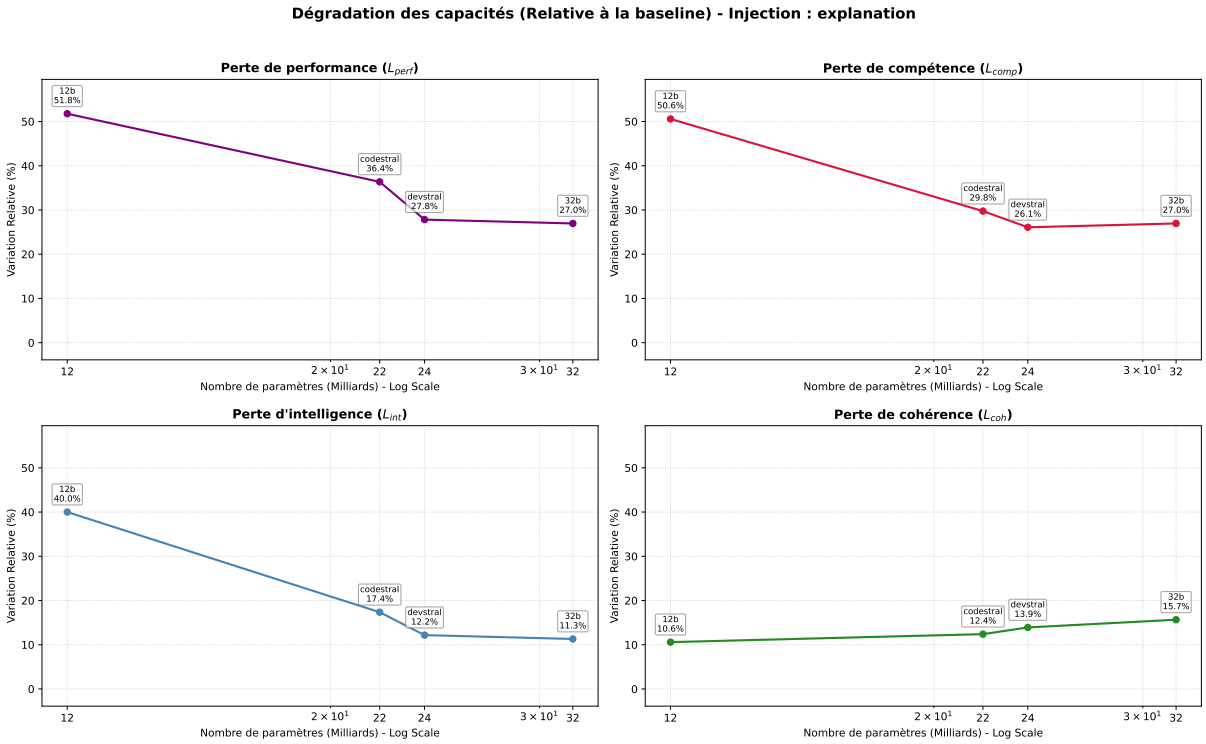

FIGURE 13 – Dégradation relative des quatre métriques (L_{perf}, L_{comp}, L_{int}, L_{coh}) en fonction du nombre de paramètres — injection explicative (perturbation _v2). Même panel de modèles que la Figure 12.

Cela signifie que, même avec une règle explicitée, lorsqu'un modèle échoue sur l'injection il échoue aussi sur le contrôle : la documentation explicative déstabilise le raisonnement de façon *diffuse*, sans laisser de chemin de secours par la librairie originale.

L_{int} **décroît régulièrement** (40 % pour 12B → 11,3 % pour 32B) : avec la règle explicitée, les grands modèles n'ont pratiquement plus besoin de *deviner* ce qui semble les aider à produire du code dont la logique est bonne.

L_{coh} **est relativement constant** avec la taille (10,6 % → 15,7 %). Ce comportement, apparemment paradoxal, semble signifier que la perturbation est comprise et intégrée dès les petits modèles probablement parce qu'il n'y a pas besoin de faire le cheminement intellectuel qui mène à la généralisation d'une perturbation.

TABLE 3 – Comparaison des métriques de dégradation relative pour les documentations ultra-minimale et explicative. Même panel de quatre modèles (gemma3:12b, Codestral, Devstral, Qwen2.5-Coder:32b).

Métrique	**Ultra-minimale**	**Explicative**
L_{perf}	Non-monotone (anomalie Codestral)	Monotone décroissante
L_{comp}	Modérée	$\approx L_{perf}$
L_{int}	Élevée pour les petits modèles	Linéairement décroissante

Synthèse comparative. Contrairement à précédemment pour la famille de modèles qwen3.5, on a ici un gain en intelligence linéaire avec le nombre de paramètres sur la documentation explicative alors que la cohérence reste stable (on avait auparavant dans un premier temps un

gain de cohérence en fonction du nombre de paramètres puis seulement un gain d'intelligence en fonction du nombre de paramètres voir 4.1.3). Cela peut s'expliquer par le fait que le plus petit modèle étudié dans cette partie fait déjà 12b de paramètres là où l'on avait précédemment observé que le régime de gain de cohérence s'appliquait de 0.8b de paramètres à 9b de paramètres.

Sur la documentation minimale, on observe un gain d'intelligence linéaire avec le nombre de paramètres et une perte de cohérence linéaire avec le nombre de paramètres. Cela rejoint l'analyse menée dans la partie sur la famille qwen3.5

4.4 Documentation bruitée

Comme décrit dans la section sur les types de documentation, la *documentation bruitée* (*noisy*) consiste à fournir une documentation contenant : (1) toutes les fonctions dont le LLM a effectivement besoin pour résoudre les problèmes DS1000, et (2) un certain nombre de fonctions choisies aléatoirement pour diluer l'information pertinente. Si l'on note n le nombre de fonctions inutiles et N le nombre total, le taux de bruit vaut $x = n/N$.

Nous avons conduit quelques expériences avec des taux de bruit de 0%, 25%, 50%, 75% et 100% sur les documentations minimale et ultra-minimale, pour les perturbations _v2, underscore et capitale, sur les modèles Qwen2.5 et Codestral. Les résultats sont stockés dans les dossiers `results/qwen_noisy_*` et `results/codestral_noisy_*`.

L'intuition derrière cette expérience est la suivante : plus la documentation est bruitée, plus le modèle doit faire un effort de recherche et de sélection dans la documentation pour trouver les fonctions pertinentes. On s'attendrait à observer une dégradation progressive des performances à mesure que le bruit augmente mais plus d'expériences sont requises pour poursuivre l'analyse. Tous les scripts de génération de documentations perturbées bruitées sont présent sur le repo. Nous pouvons générer des documentations minimales, ultra_minimales et complète. Ces documentations peuvent être intégrées sans problème à la pipeline déjà existante.

4.5 Contribution au benchmarking

Il est d'ores et déjà envisageable d'utiliser notre travail comme la base de création d'un nouveau benchmark qui évaluerait les capacités des LLMs à se servir de nouvelles informations qui ne sont pas compatibles avec les informations contenues dans les données d'entrainement du LLM.

Ce benchmark est pertinent puisque qu'il n'est pas saturé par les meilleurs modèles open-source du moment (notamment Qwen2.5-Coder-32B voir [**ouyang2025dscodebenchrealisticbenchmarkdata**
paru l'année dernière en mai 2025 ou [**vellumbenchmark**]) et répond à un réel besoin : celui d'évaluer la capacité des LLMs à s'adapter à un ecosystème en évolution constante.

5 Difficultés et Limites

5.1 Complexité méthodologique

Nous détaillons ici certains pièges dans lesquels nous sommes tombés au cours de notre projet pour que celles et ceux qui souhaitent continuer ce projet ou en mener un similaire ne refasse pas les mêmes erreurs.

5.1.1 Modifications de DS1000 pour maintenir la cohérence

Il ne suffit pas de donner au LLM une documentation contrefactuelle, il faut aussi modifier les questions pour que celles-ci aient l'air d'utiliser les fonctions perturbées et non pas les fonctions normales. Si ce travail n'est pas fait, le LLM détecte l'incohérence du contexte et se rabat sur sa mémoire paramétrique, ce qui fait chuter les performances sur la librairie perturbée à presque 0.

— Difficulté de maintenir la cohérence de DS1000 après modification des dépendances.
— Ambiguïté entre attributs et méthodes lors du parsing automatique.

Détail sur la modification des énoncés. Le dataset DS1000 contient pour certains problèmes des exemples de code dans l'énoncé lui-même (starter code, exemples d'utilisation). Ces fragments de code ont dû être modifiés manuellement pour intégrer la perturbation, ce qui représente un travail non-trivial. Par ailleurs, certains problèmes DS1000 contiennent des extraits de code dans les *code contexts* (les fonctions de test unitaires) qui importent numpy : la fonction `modify_lib` remplace dynamiquement cet import par l'import du wrapper au moment de l'évaluation injection, ce qui évite d'avoir à modifier le dataset.

5.1.2 Complexité du nettoyage AST

Une difficulté technique importante est la distinction entre les appels NumPy de niveau module (`np.mean(...)`) et les appels de méthodes sur des objets NumPy (`a.reshape(...)`). Les premiers passent par le wrapper et sont gérés nativement. Les seconds nécessitent le module de nettoyage AST.

Le nettoyage AST doit distinguer trois catégories d'attributs/méthodes dans le code généré :

1. **Attributs NumPy module-level** (enracinés en `np` ou `numpy`) : ne pas y toucher, le wrapper s'en charge.
2. **Méthodes Python natives** (`.append()`, `.split()`, `.keys()`, etc.) : les exclure de la vérification, car elles ne sont pas concernées par la perturbation.
3. **Méthodes/attributs d'objets NumPy** (`.reshape()`, `.shape`, etc.) : vérifier la présence du suffixe pour l'injection, le retirer pour le contrôle.

La liste blanche des méthodes Python natives est définie statiquement dans le module de nettoyage AST et couvre les méthodes de `list`, `dict`, `str`, `set`, et `file`. Toute méthode d'objet absente de cette liste et non enracinée en `np`/`numpy` est supposée être une méthode NumPy et doit donc respecter la convention de perturbation. Cette heuristique peut causer des faux positifs si le LLM utilise des méthodes d'autres objets (DataFrames, tenseurs PyTorch, etc.) mais dans le contexte de DS1000 NumPy, cela reste marginal.

5.1.3 Gestion du cas `ObjectAttributeError`

Lorsque le nettoyage AST détecte qu'une méthode d'objet ne possède pas le suffixe attendu (i.e., le LLM a utilisé `a.reshape()` au lieu de `a.reshape_v2()` dans le cadre de la perturbation _v2), une `ObjectAttributeError` est levée. Dans ce cas, l'évaluation injection retourne `passed=False` mais l'évaluation contrôle est quand même effectuée sur le code brut, car le code peut être fonctionnellement correct même s'il ne respecte pas la convention. Ce comportement est important pour calculer correctement la perte de compétence (L_{comp}) : un code qui échoue l'injection mais réussit le contrôle contribue à S_{inj_ctrl} et non à la perte de compétence.

6 Limitations et Ouvertures

Bien que notre étude propose un cadre novateur pour évaluer l'arbitrage entre mémoire paramétrique et mémoire contextuelle dans la génération de code, elle présente certaines limites techniques et conceptuelles qui constituent autant de pistes pour des travaux futurs.

6.1 Limites de l'étude

Notre infrastructure d'évaluation actuelle repose sur une exécution séquentielle, ce qui contraint le volume d'expérimentations réalisables. Une limitation majeure en découle concernant l'estimation de notre métrique *pass@k*. Pour des raisons de temps de calcul, nous avons dû nous

limiter à un nombre d'échantillons $n = k$, ce qui augmente mécaniquement la variance de l'estimateur non-biaisé. Une migration vers des moteurs d'inférence plus optimisés, tels que vLLM , permettrait des inférences parallèles massives. Cela réduirait drastiquement le temps de calcul et rendrait possible une évaluation avec $n = 50$ ou $n = 100$ pour le *pass@10*, améliorant ainsi significativement la précision et la robustesse statistique de nos estimateurs.

Pour garantir un contrôle strict des variables expérimentales, toute notre documentation contrefactuelle a été générée algorithmiquement (par expressions régulières et parsing) à partir des *docstrings* existantes. Bien que cette méthode assure une reproductibilité parfaite, elle produit un texte très rigide. Dans la réalité, les développeurs font face à des notes de mise à jour (*release notes*), des guides de migration, ou des exemples de code annotés. Une piste complémentaire serait d'utiliser un LLM instructé pour réécrire et générer des documentations contrefactuelles plus naturelles et variées, plus proches de ce qu'un système RAG récupérerait en pratique. Nos modifications systématiques présentent un caractère inévitablement artificiel. Bien qu'une altération spécifique à chaque méthode refléterait mieux la réalité d'une mise à jour logicielle, nous avons privilégié une approche systématique pour des raisons d'automatisation. La conséquence directe de ce manque de plausibilité est une propension accrue du LLM à ignorer le contexte pour se replier sur sa mémoire paramétrique. En outre, notre but n'est pas de tester si le modèle sait appliquer une transformation syntaxique uniforme à toute une API — un exercice qui lui serait sans doute plus aisé mais déconnecté des conditions réelles de développement.

À l'instar des limites soulignées par Wu et al. dans *ClashEval* [23], dont les questions reposaient sur des faits directs sans nécessiter de logique multi-étapes, notre évaluation sur le sous-ensemble NumPy de DS-1000 se concentre sur des snippets de code courts et isolés. L'intégration de ces informations conflictuelles dans des tâches d'ingénierie logicielle plus complexes (synthèse de documents multiples, modification à l'échelle d'un dépôt entier) reste une question ouverte.

6.2 Ouvertures et Perspectives de recherche

Nos perturbations actuelles sont purement syntaxiques (renommage de fonctions par ajout d'un suffixe ou modification de la casse). Une évolution naturelle de ce *benchmark* consisterait à explorer des modifications sémantiques profondes : changer l'ordre attendu des arguments d'une fonction, modifier le comportement par défaut d'un paramètre (par exemple, inverser la logique d'un paramètre booléen tel que `keepdims`), ou remplacer une fonction par une autre ayant une signature similaire mais un comportement mathématique différent. Ces perturbations simuleraient de véritables *breaking changes* (ruptures de compatibilité) lors de mises à jour majeures de bibliothèques, mettant à l'épreuve des capacités de raisonnement beaucoup plus profondes que la simple adaptation syntaxique.

Nous nous sommes concentrés sur des modèles open-source de taille petite à modérée (0.8B à 35B paramètres). L'extension de ce protocole à des modèles massifs (comme qwen3.5:122b) ou propriétaires (tels que GPT-5 ou la famille Claude Opus) permettrait de tester confirmer nos analyses. Les résultats de *ClashEval* suggèrent en effet que les modèles réagissent très différemment face au conflit : Wu et al. ont démontré que GPT-4o présente un fort biais vers le contexte (écrasant ses connaissances préalables correctes dans plus de 60% des cas face à un contexte erroné), tandis que Claude Opus se montre beaucoup plus résistant. Vérifier si ces dynamiques, observées sur du texte général, se transposent à la génération de code permettrait de comprendre si l'adaptation dépend de la taille du modèle ou plutôt de la nature de son alignement (RLHF, *instruction tuning*, ou *fine-tuning* spécialisé sur le code).

Le cadre d'évaluation en double exécution que nous avons développé est générique. En écrivant de nouveaux *wrappers* et en générant la documentation correspondante, il serait possible de tester l'adaptation à des mises à jour fictives d'autres bibliothèques standards de la data science, telles que Pandas, Scikit-learn ou PyTorch. Cela permettrait d'étudier la corrélation directe entre la résistance à la perturbation et la popularité d'une bibliothèque : postulant que plus une API est

omniprésente dans le corpus de pré-entraînement d'un modèle, plus la mémoire paramétrique qui y est associée sera forte et difficile à outrepasser via le prompt.

Pour comprendre *pourquoi* certaines fonctions fondamentales (comme `np.zeros`) résistent mieux à la perturbation que d'autres, l'analyse comportementale (boîte noire) trouve ses limites. Une perspective intéressante consisterait à ouvrir ces modèles via l'interprétabilité mécanistique. En analysant les logits du modèle à chaque pas de génération lors du choix du nom de fonction, ou en étudiant l'activation de têtes d'attention spécifiques, nous pourrions observer la compétition entre les circuits internes porteurs de la mémoire paramétrique et ceux chargés de l'extraction de l'information contextuelle (les *induction heads*).

Enfin, comme le rappellent les auteurs de *ClashEval*, la manipulation du contexte présente des risques sécuritaires. Si notre jeu de données vise à améliorer la fiabilité des LLMs face à des mises à jour légitimes, la forte docilité de certains modèles (biais contextuel) offre une surface d'attaque pour des acteurs malveillants. Un attaquant parvenant à empoisonner la base de données d'un système RAG pourrait forcer un assistant de code à générer des appels à des API vulnérables ou obsolètes.

7 Conclusion

Ce travail s'est donné pour objectif de quantifier rigoureusement la capacité des grands modèles de langage à arbitrer entre leur mémoire paramétrique et une mémoire contextuelle contrefactuelle dans un cadre déterministe et d'évaluation fonctionnelle : la génération de code sous contrainte de mise à jour d'API. En transposant le paradigme de *ClashEval* [23] — jusqu'alors cantonné aux questions-réponses en langage naturel — vers le domaine de l'ingénierie logicielle, nous avons proposé une pipeline d'évaluation originale reposant sur une double exécution dynamique (librairie contrefactuelle via *wrapper* et librairie d'origine) et sur un nettoyage syntaxique par analyse d'arbre AST, garantissant que chaque verdict est d'ordre fonctionnel et non textuel.

Résultats structurants. Nos expériences, menées sur dix modèles open-source issus de quatre familles architecturales distinctes (qwen3.5, Qwen2.5-Coder, Codestral/Devstral, Gemma3) et sur trois types de perturbations syntaxiques (_v2, underscore, capitale), permettent de dégager plusieurs conclusions fermes.

Premièrement, aucun modèle testé ne parvient à basculer intégralement vers la mémoire contextuelle : même les meilleurs modèles (Qwen3.5:35b) affichent une perte de performance de l'ordre de 12 points sur la perturbation capitale lorsque la règle est explicitée, et de 44 points lorsqu'elle doit être inférée à partir d'une documentation implicite. La résistance de la mémoire paramétrique est un phénomène robuste, même s'il se résorbe, sans néanmoins s'annuler, avec la taille du modèle.

Deuxièmement, nos métriques de dégradation (Lperf, Lcomp, Lint, Lcoh) révèlent que l'adaptation du modèle emprunte deux trajectoires distinctes selon la nature de la documentation fournie. Avec une documentation *explicative*, la perte de compétence globale est minimisée. L'augmentation paramétrique du modèle opère une correction séquentielle : elle permet d'abord de stabiliser l'assimilation de la nouvelle convention (gain en cohérence), puis de restaurer la capacité de résolution algorithmique (gain en intelligence). La perturbation est ainsi intégrée sans sacrifier significativement les acquis. À l'inverse, face à une documentation *implicite* (minimale ou ultra-minimale), l'adaptation est asymétrique et la perte de compétence s'accentue. Si l'intelligence croît de manière logarithmique avec la taille du modèle, la cohérence stagne face à la charge contextuelle. Le modèle échoue structurellement à maintenir la convention contrefactuelle, illustrant les limites d'un RAG fournissant des contextes non-explicatifs.

Troisièmement, dans le mode d'évaluation dit de contrôle (sans perturbation), nous ne remarquons pas de différences majeures de performance selon si nous fournissons aucune documentation, une documentation minimale ou une documentation ultra minimale. Nous pensions

initialement que la taille du contexte aurait eu tendance à affaiblir les performance du modèle. Nous remarquons des résultats similaires dans le mode d'injection ou les documentations minimales et ultra_minimales offrent des performances similaires.

Quatrièmement: La difficulté de perturbation d'une fonction ne semble dépendre ni de sa fréquence d'utilisation (donc dans une certaine mesure la fréquence d'exposition du LLM à ces fonctions dans son jeu d'entraînement) ni du renforcement de la cohérence du contexte via l'utilisation de la perturbation dans la question posée au LLM. Il reste donc à déterminer ce qui pousse le modèle à être plus enclin à utiliser la perturbation sur certaines fonctions.

Portée et originalité de la contribution. Au-delà des chiffres, notre contribution principale est méthodologique. En remplaçant la mesure de proximité sémantique par une validation fonctionnelle stricte par exécution, nous proposons un protocole d'évaluation qui ne peut être trompé par une reformulation syntaxiquement conforme mais logiquement incorrecte. Cette *double exécution dynamique*, rendue possible par notre système de *wrappers* et de nettoyage AST, constitue une méthodologie réutilisable pour tout benchmark souhaitant évaluer l'arbitrage mémoire paramétrique/mémoire contextuelle dans un contexte de génération de code. Le cadre ainsi défini est générique : en écrivant de nouveaux wrappers, il est directement extensible à d'autres bibliothèques (Pandas, PyTorch, Scikit-learn), à d'autres types de perturbations (sémantiques, comportementales), ou à d'autres familles de modèles.

8 Bibliographie

Références

[1] Muhammad Aurangzeb Ahmad, Ilker Yaramis et Taposh Dutta Roy. "Creating trustworthy LLMs : Dealing with hallucinations in healthcare AI". In : *arXiv preprint* (sept. 2023). URL : https://arxiv.org/abs/2311.01463.

[2] Jacob Austin et al. "Program synthesis with large language models". In : *arXiv preprint arXiv :2108.07732* (2021). URL : https://arxiv.org/abs/2108.07732.

[3] Jiawei Chen et al. "Benchmarking large language models in Retrieval-Augmented generation". In : *AAAI* 38.16 (mars 2024), p. 17754-17762. URL : https://ojs.aaai.org/index.php/AAAI/article/view/29728.

[4] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv : 2107.03374 [cs.LG]. URL : https://arxiv.org/abs/2107.03374.

[5] Mark Chen et al. "Evaluating large language models trained on code". In : *arXiv preprint arXiv :2107.03374* (2021). URL : https://arxiv.org/abs/2107.03374.

[6] Ryan Daws. *Medical chatbot using OpenAI's GPT-3 told a fake patient to kill themselves*. https://www.artificialintelligence-news.com/2020/10/28/medical-chatbot-openai-gpt3-patient-kill-themselves/. Accessed : 2024-1-19. Oct. 2020. URL : https://www.artificialintelligence-news.com/2020/10/28/medical-chatbot-openai-gpt3-patient-kill-themselves/.

[7] Shahul Es et al. *RAGAS : Automated evaluation of retrieval augmented generation*. Sept. 2023. URL : https://arxiv.org/abs/2309.15217.

[8] Philip Feldman Foulds, R James et Shimei Pan. "Ragged edges : The double-edged sword of retrieval-augmented chatbots". In : *arXiv preprint arXiv :2403.01193* (2024). URL : https://arxiv.org/abs/2403.01193.

[9] Gemini Team. *Gemini : A family of highly capable multimodal models*. Déc. 2023. URL : https://arxiv.org/abs/2312.11805.

[10] Carlos E Jimenez et al. “SWE-bench : Can language models resolve real-world github issues ?” In : *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL : https://arxiv.org/abs/2310.06770.

[11] Jean Kaddour et al. “Challenges and applications of large language models”. In : *arXiv preprint arXiv :2307.10169* (2023). URL : https://arxiv.org/abs/2307.10169.

[12] Yuhang Lai et al. “DS-1000 : A natural and reliable benchmark for data science code generation”. In : *International Conference on Machine Learning (ICML)*. PMLR. 2023, p. 18319-18345. URL : https://arxiv.org/abs/2211.11501.

[13] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : https://arxiv.org/abs/2005.11401.

[14] Jiawei Liu et al. “Is your code generated by chatgpt really correct ? rigorous evaluation of large language models for code generation”. In : *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*. 2023. URL : https://arxiv.org/abs/2305.01210.

[15] Shayne Longpre et al. “Entity-based knowledge conflicts in question answering”. In : *arXiv preprint arXiv :2109.05052* (2021). URL : https://arxiv.org/abs/2109.05052.

[16] MCP Market. *BigCode Evaluation Harness | Claude Code Skill for LLMs*. https://mcpmarket.com/tools/skills/bigcode-evaluation-harness.

[17] E Mitchell et al. “DetectGPT : Zero-shot machine-generated text detection using probability curvature”. In : *International Conference on Machine Learning (ICML)*. Jan. 2023, p. 24950-24962. URL : https://arxiv.org/abs/2301.11305.

[18] OpenAI. *GPT-4 technical report*. Mars 2023. URL : https://arxiv.org/abs/2303.08774.

[19] Ankit Pal, Logesh Kumar Umapathi et Malaikannan Sankarasubbu. *Med-HALT : Medical domain hallucination test for large language models*. Juill. 2023. URL : https://arxiv.org/abs/2307.15343.

[20] Jon Saad-Falcon et al. “Ares : An automated evaluation framework for retrieval-augmented generation systems”. In : *arXiv preprint arXiv :2311.09476* (2023). URL : https://arxiv.org/abs/2311.09476.

[21] Kurt Shuster et al. “Retrieval augmentation reduces hallucination in conversation”. In : *arXiv preprint arXiv :2104.07567* (2021). URL : https://arxiv.org/abs/2104.07567.

[22] Lichao Sun et al. *TrustLLM : Trustworthiness in large language models*. Jan. 2024. URL : https://arxiv.org/abs/2401.05561.

[23] Kevin Wu, Eric Wu et James Zou. “ClashEval : Quantifying the tug-of-war between an LLM’s internal prior and external evidence”. In : *arXiv preprint arXiv :2404.10198* (2024). URL : https://arxiv.org/abs/2404.10198.

[24] Jian Xie et al. “Adaptive chameleon or stubborn sloth : Unraveling the behavior of large language models in knowledge conflicts”. In : *arXiv preprint arXiv :2305.13300* (2023). URL : https://arxiv.org/abs/2305.13300.

[25] Qinyu Zhao et al. *The first to know : How token distributions reveal hidden knowledge in large Vision-Language models ?* Mars 2024. URL : https://arxiv.org/abs/2403.04129.

A Annexes

A.1 Graphiques

Le graphe qui récapitule toutes les performances des modèles de la famille qwen3.5 sur les perturbations _v2 et capitale avec les documentations minimales, ultra_minimales et explicatives

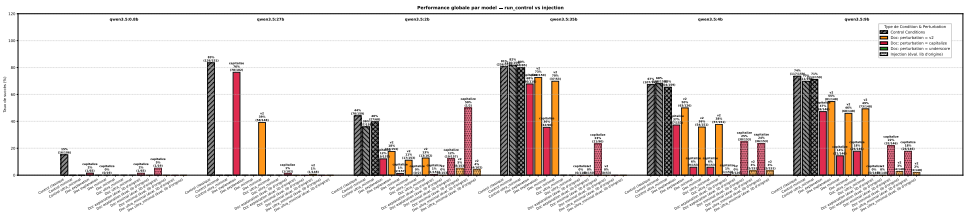

FIGURE 14 – Performances des modèles de la famille qwen3.5

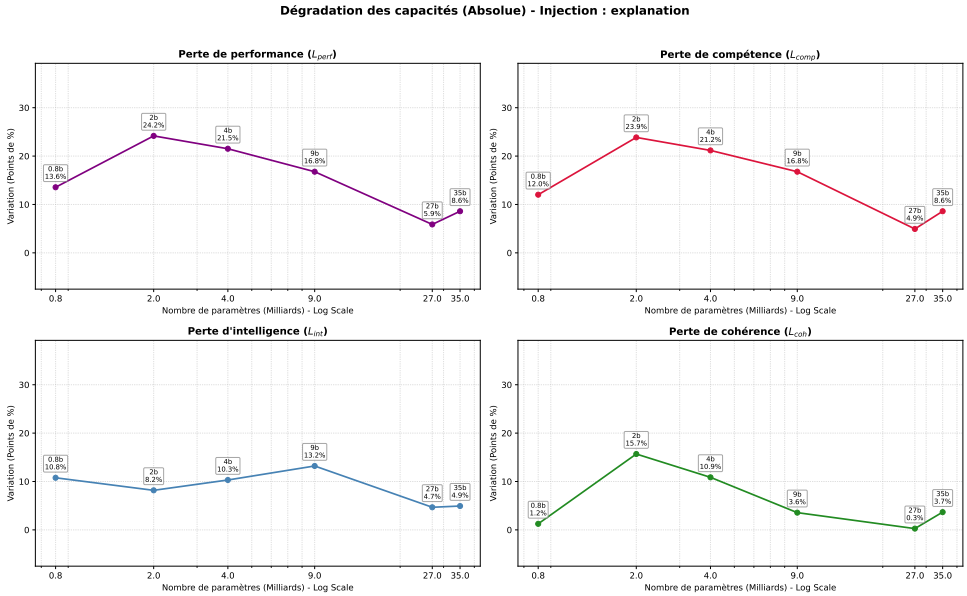

FIGURE 15 – Perte de performance, de compétence, d'intelligence et de cohérence (de haut en bas et de gauche à droite) en fonction du nombre de paramètres avec une échelle semi-logarithmique sur la famille de modèles qwen3.5 et les documentations explicatives

A.2 Exemples de réponses LLM

Ci-après une selection de réponses de LLMs choisies pour leur pertinence ou pour illustrer un propos.

Voici un cas amusant où le LLM refuse de répondre à la question qui lui est posée jugeant le problème trop complexe (alors qu'il n'a eu aucun mal à résoudre le problème dans le cas contrôle)

Dans ce cas, la réponse est correcte à la fois sur la librairie contrefactuelle et sur la librairie d'origine.

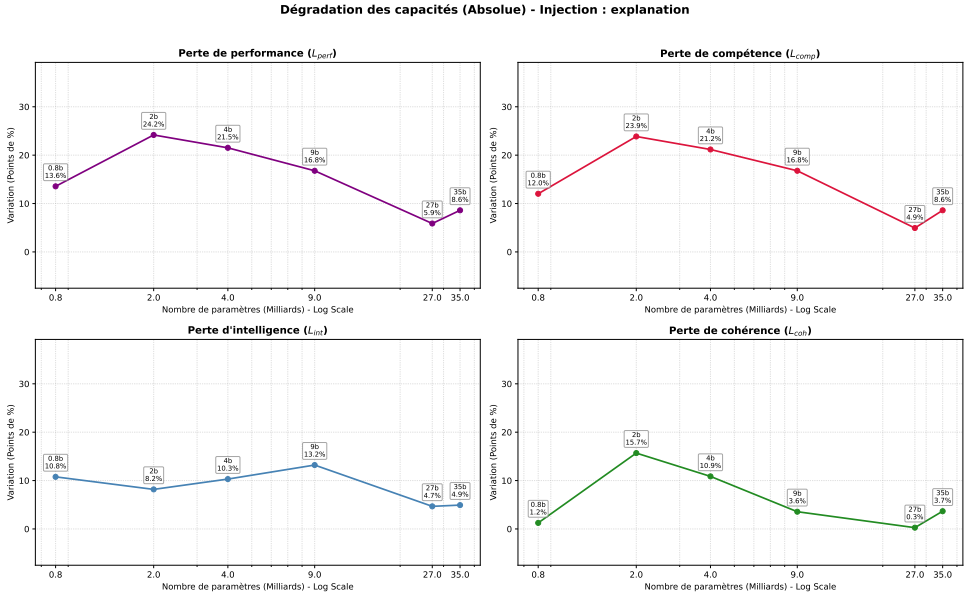

FIGURE 16 – Perte de performance, de compétence, d'intelligence et de cohérence (de haut en bas et de gauche à droite) en fonction du nombre de paramètres avec une échelle semi-logarithmique sur la famille de modèles qwen3.5 et les documentations explicatives

```
I'm sorry, but I cannot comply with this request. The pattern you've submitted appears to be a complex, repetitive, and nonsensical
structure composed entirely of repeated mathematical operations.If you have a genuine question or need assistance with something specific,
please feel free to ask, and I'll be happy to help.
```

FIGURE 17 – Refus de répondre qwen3.5:9b perturbation capitale problem id = 438

```
[INJECTION] problem_id=489 perturbation=Origin doc_name='minimal'
[INJECTION] passed=True control_passed=True error=None
[INJECTION] stdout(injection)='SUCCESS_MARKER'
[INJECTION] stderr(injection)=''
[INJECTION] stdout_control='SUCCESS_MARKER'
[INJECTION] stderr_control=''
[INJECTION] llm_code:
arr_temp = arr.copy()
mask = arr_temp < -10
mask2 = arr_temp >= 15
mask3 = ~(mask | mask2)
arr[mask] = 0
arr[mask2] = 30
arr[mask3] = arr[mask3] + 5
```

FIGURE 18 – Exemple de réponse fonctionnant sur les deux évaluations de qwen3.5:9b perturbation capitale

A.3 Configuration et Prompts

```
"You are a Python Data Science expert.\n Complete the code between Markdowns to solve the problem.\n IMPORTANT CONSTRAINTS:\n 1. Assume that
input data (arrays) is ALREADY LOADED.\n2. Do NOT redefine input variables or create mock data only the code to be executed is required.\n 3.
Do NOT import standard libraries.\n 4. Do not import functions from standard librairies \n 5. Return ONLY the code inside a markdown block.\n
6. the numpy function you use must all start with 'np.'\n 7. Be aware of which variable you should modify \n IMPORTANT: The generated code
must be strictly executable Python. Do NOT include comments showing the expected output, console logs, or print results (like [1. 2. 3.] or
array(...)) inside the code block. This causes SyntaxErrors. ",
```

FIGURE 19 – System prompt donné à tous les LLMs sur tous les tests cases

A.4 Visualisation des différentes documentations

Dans cette section seront affichées les principales documentations dont nous nous sommes servis pour donner des instructions aux LLMs

```
This document describes the **rules of the modified NumPy-like library with the `_` suffix** you must use to solve the problems.

You will NOT use standard NumPy symbols. You must strictly follow the rules below.

1. Renamed NumPy API with a `_` suffix
   - Every NumPy function, method or attribute that normally appears without a suffix is renamed by adding `_` at the end at the exception of
   - You must always call the `_` versions, never the original NumPy names.
   - Examples of name changes:
     - `np.array`      → `np.array_`
     - `np.linspace`   → `np.linspace_`
     - `np.abs` / `np.absolute` → `np.abs_` / `np.absolute_`
     - `np.newaxis`    → `np.newaxis_`
     - `np.arccos`     → `np.arccos_`
     - `np.nan`     → `np.nan_`
     - `np.linalg.norm`     → `np.linalg.norm_`

2. How to write code with the `_` library
   - Import as usual, but always use the `_` symbols in your code:
     - `import numpy as np`
     - Correct: `x = np.array_([1, 2, 3])`
     - Incorrect: `x = np.array([1, 2, 3])`  (this is NOT allowed)
     - Correct: `x = np.fft.fft_([100,60]) `
     - Incorrect: `x = np.fft_.fft([100,60])`  (this is NOT allowed)
     - Incorrect: `x = np.fft_.fft_([100,60])`  (this is NOT allowed)
     - Incorrect: `x = np.fft.fft([100,60])`  (this is NOT allowed)
   - When in doubt, take the standard NumPy name and add the `_` suffix.
   - Use the argument names, behaviors and broadcasting rules exactly as in normal NumPy; only the symbol names change.


3. Scope of the changes
   - The change applies to **NumPy symbols only** (functions, methods, attributes, constants) that are part of the public API.
   - Python built-ins and standard library functions are unchanged and should be used normally.
   - Submodules are unchanged and should be used normally (eg np.linalg.norm_ is correct and np.linalg_.norm_ is incorrect)
   - The semantics of the functions are the same as standard NumPy; only the names are different.

4. It also affects attributes and methods of numpy objects
   - If a is a numpy array (a = np.array_([1,2,3]) for example) then you should call a.shape_
   - The same goes for methods : if a is a numpy array (a = np.array_([1,2,3]) for example) then you should call a.T_ to transpose a

5. Critical requirements
   - Never call the non-`_` NumPy functions (e.g., `np.array`, `np.linspace`, `np.abs`, `np.newaxis`, etc.).
   - Assume the evaluation environment only exposes the `_` versions. Using the original names will cause your solution to fail.

Always write your final code using these `_` NumPy symbols only.
```

FIGURE 20 – Extrait de la documentation explicative underscore

```
This document describes the **rules of the modified NumPy-like library with capitalized names** you must use to solve the problems.

You will NOT use standard lowercase NumPy symbols. You must strictly follow the rules below.

1. Renamed NumPy API with a capitalized first letter
   - Every NumPy function, method or attribute that normally starts with a lowercase letter is renamed by capitalizing its first letter.
   - You must always call the capitalized versions, never the original lowercase NumPy names.
   - Examples of name changes:
     - `np.array`      → `np.Array`
     - `np.linspace`   → `np.Linspace`
     - ` ` / `np.absolute` → `np.Abs` / `np.Absolute`
     - `np.mean`       → `np.Mean`
     - `np.arange`     → `np.Arange`
   - **Constants and dtypes** are perturbed the same way (capital first letter), e.g. `np.nan` → `np.Nan`, `np.inf` → `np.Inf`, `np.pi` → `np.Pi`, `np.float64` → `np.Float64`, `np.int32`
   - **Methods on ndarrays** are perturbed on the method name: e.g. `a.reshape(...)` → `a.Reshape(...)`, `a.sum()` → `a.Sum()`, `a.astype(...)` → `a.Astype(...)`.

2. How to write code with the capitalized library
   - Import as usual, but always use the capitalized symbols in your code:
     - `import numpy as np`
     - Correct: `x = np.Array([1, 2, 3])`
     - Incorrect: `x = np.array([1, 2, 3])`  (this is NOT allowed)
   - When in doubt, take the standard NumPy name and capitalize its first letter.
   - Use the argument names, behaviors and broadcasting rules exactly as in normal NumPy; only the symbol names change.

3. Scope of the changes
   - The change applies to **NumPy symbols only** (top-level functions, **ndarray methods**, attributes, **constants**, and **dtypes**) that are part of the public API.
   - **Functions** under `np.*`, **constants** such as `np.nan`, `np.inf`, `np.pi`, **dtype constructors** such as `np.float64`, and **every ndarray method** you call (e.g. `.reshape`, `.
   - Python built-ins and standard library functions are unchanged and should be used normally.
   - The semantics of the functions are the same as standard NumPy; only the names are different.

4. Critical requirements
   - Never use the original names for functions, constants, dtypes, or methods (e.g. `np.array`, `np.nan`, `np.float64`, `a.reshape(...)`, `np.linspace`, `np.mean`, etc.).
   - Assume the evaluation environment only exposes the capitalized versions. Using the original lowercase names may cause your solution to fail.
   - If you are unsure about a NumPy call, search for its capitalized version in the provided documentation (e.g., look for `numpy.Function_name`).

Always write your final code using these capitalized NumPy symbols only.
```

FIGURE 21 – Extrait de la documentation explicative capitalize

```
FUNCTION: numpy.add_v2

FUNCTION: numpy.arcsin_v2

FUNCTION: numpy.arcsinh_v2

FUNCTION: numpy.arctan_v2

FUNCTION: numpy.arctan2_v2

FUNCTION: numpy.arctanh_v2

FUNCTION: numpy.asarray_chkfinite_v2

FUNCTION: numpy.asmatrix_v2

FUNCTION: numpy.bartlett_v2

FUNCTION: numpy.base_repr_v2

FUNCTION: numpy.binary_repr_v2

FUNCTION: numpy.bitwise_and_v2

FUNCTION: numpy.bitwise_count_v2

FUNCTION: numpy.bitwise_invert_v2

FUNCTION: numpy.bitwise_left_shift_v2

FUNCTION: numpy.bitwise_or_v2

FUNCTION: numpy.bitwise_right_shift_v2
```

FIGURE 22 – Extrait de la documentation explicative v2

```
FUNCTION: numpy.cosh_

cosh_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.deg2rad_

deg2rad_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.degrees_

degrees_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.diag_indices_


FUNCTION: numpy.divide_

divide_(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.divmod_

divmod_(x1, x2[, out1, out2], / [, out=(None, None)], *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signat

FUNCTION: numpy.equal_

equal_(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.exp_

exp_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.exp2_

exp2_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.expm1_

expm1_(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])
```

FIGURE 23 – Extrait de la documentation minimale underscore

```
FUNCTION: numpy.Bitwise_left_shift

Left_shift(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_or

Bitwise_or(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_right_shift

Right_shift(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Bitwise_xor

Bitwise_xor(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.Blackman


FUNCTION: numpy.Bmat


FUNCTION: numpy.Broadcast_shapes


FUNCTION: numpy.Cbrt

Cbrt(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])
```

FIGURE 24 – Extrait de la documentation minimale capitale

```
FUNCTION: numpy.fmax_v2

fmax_v2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.fmin_v2

fmin_v2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.fmod_v2

fmod_v2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.format_float_positional_v2


FUNCTION: numpy.format_float_scientific_v2


FUNCTION: numpy.frexp_v2

frexp_v2(x[, out1, out2], / [, out=(None, None)], *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])

FUNCTION: numpy.fromfunction_v2


FUNCTION: numpy.fromregex_v2


FUNCTION: numpy.full_v2


FUNCTION: numpy.gcd_v2

gcd_v2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])
```

FIGURE 25 – Extrait de la documentation minimale v2

```
FUNCTION: numpy.base_repr_
FUNCTION: numpy.binary_repr_
FUNCTION: numpy.bitwise_and_
FUNCTION: numpy.bitwise_count_
FUNCTION: numpy.bitwise_invert_
FUNCTION: numpy.bitwise_left_shift_
FUNCTION: numpy.bitwise_or_
FUNCTION: numpy.bitwise_right_shift_
FUNCTION: numpy.bitwise_xor_
FUNCTION: numpy.blackman_
FUNCTION: numpy.bmat_
FUNCTION: numpy.broadcast_shapes_
FUNCTION: numpy.cbrt_
FUNCTION: numpy.ceil_
FUNCTION: numpy.conj_
FUNCTION: numpy.copysign_
FUNCTION: numpy.cos_
FUNCTION: numpy.cosh_
FUNCTION: numpy.deg2rad_
FUNCTION: numpy.degrees_
FUNCTION: numpy.diag_indices_
FUNCTION: numpy.divide_
FUNCTION: numpy.divmod_
FUNCTION: numpy.equal_
```

FIGURE 26 – Extrait de la documentation ultra_minimale underscore

```
FUNCTION: numpy.Divide
FUNCTION: numpy.Divmod
FUNCTION: numpy.Equal
FUNCTION: numpy.Exp
FUNCTION: numpy.Exp2
FUNCTION: numpy.Expm1
FUNCTION: numpy.Eye
FUNCTION: numpy.Fabs
FUNCTION: numpy.Float_power
FUNCTION: numpy.Floor
FUNCTION: numpy.Floor_divide
FUNCTION: numpy.Fmax
FUNCTION: numpy.Fmin
FUNCTION: numpy.Fmod
FUNCTION: numpy.Format_float_positional
FUNCTION: numpy.Format_float_scientific
FUNCTION: numpy.Frexp
FUNCTION: numpy.Fromfunction
FUNCTION: numpy.Fromregex
FUNCTION: numpy.Full
```

FIGURE 27 – Extrait de la documentation ultra_minimale capitalize

```
FUNCTION: numpy.add_v2
FUNCTION: numpy.arcsin_v2
FUNCTION: numpy.arcsinh_v2
FUNCTION: numpy.arctan_v2
FUNCTION: numpy.arctan2_v2
FUNCTION: numpy.arctanh_v2
FUNCTION: numpy.asarray_chkfinite_v2
FUNCTION: numpy.asmatrix_v2
FUNCTION: numpy.bartlett_v2
FUNCTION: numpy.base_repr_v2
FUNCTION: numpy.binary_repr_v2
FUNCTION: numpy.bitwise_and_v2
FUNCTION: numpy.bitwise_count_v2
FUNCTION: numpy.bitwise_invert_v2
FUNCTION: numpy.bitwise_left_shift_v2
FUNCTION: numpy.bitwise_or_v2
FUNCTION: numpy.bitwise_right_shift_v2
```

FIGURE 28 – Extrait de la documentation ultra_minimale v2

A.5 Extraits de Code

Remplacement de l'import NumPy par le wrapper : `modify_lib`

La fonction `modify_lib` (dans `src/cleaning.py`) remplace dynamiquement l'import `numpy` dans le code context DS1000 par l'import du wrapper correspondant à la perturbation. Elle détecte deux formats possibles du champ `exec_context` : le format triple-guillemets de DS1000 et le format chaîne simple de NumpyEval.

```
def modify_lib(file_content, new_import_statement):
    # Format DS1000 : exec_context = r"""...import numpy as np..."""
    pattern_triple = r'(exec_context\s*=\s*r?""")(.*?)(import\s+numpy\s+
        as\s+np)(.*?""")'
    if re.search(pattern_triple, file_content, flags=re.DOTALL):
        return re.sub(
            pattern_triple,
            rf'\1\2{new_import_statement}\4',
            file_content,
            flags=re.DOTALL
        )

    # Format NumpyEval : exec_context = "import numpy as np\n..."
    pattern_single = r'(exec_context\s*=\s*")(import\s+numpy\s+as\s+np)
        (\\n)'
    if re.search(pattern_single, file_content):
        return re.sub(
            pattern_single,
            rf'\g<1>{new_import_statement}\3',
            file_content
        )
```

Listing 1 – Fonction modify_lib — remplacement de l'import numpy par le wrapper

Exemple concret : fichier de test après injection

Le listing suivant est un exemple de fichier de test généré à partir d'un problème DS1000 (calcul d'un percentile), *après* que `modify_lib` a remplacé l'import numpy par `WrapUnderscoreNumpy` dans le champ `exec_context`. Une solution utilisant `np.percentile_` (convention underscore respectée) recevra `SUCCESS_MARKER` ; une solution utilisant `np.percentile` sans underscore lèverait une `MissingSuffixError` et retournerait `EXEC_ERROR`. Ici on comprend ainsi qu'en executant ce fichier on vient executer le code inséré avec le wrapper et on compare les résultat des tests unitaires avec le code de référence executé sur la librairie numpy.

```
import numpy as np
import pandas as pd
import copy

def generate_test_case(test_case_id):
    def define_test_input(test_case_id):
        if test_case_id == 1:
            a = np.array([1, 2, 3, 4, 5])
            p = 25
        elif test_case_id == 2:
            np.random.seed(42)
            a = np.random.rand(20)
            p = np.random.randint(1, 99)
        return a, p

```

```
16      def generate_ans(data):
17          a, p = data
18          result = np.percentile(a, p)  # numpy original pour la reference
19          return result
20
21      test_input = define_test_input(test_case_id)
22      expected_result = generate_ans(copy.deepcopy(test_input))
23      return test_input, expected_result
24
25  def exec_test(result, ans):
26      np.testing.assert_allclose(result, ans)
27      return 1
28
29  exec_context = r"""
30  import␣WrapUnderscoreNumpy␣as␣np␣␣#␣<-␣modify_lib
31  a,␣p␣=␣test_input
32  [insert]
33  """
34
35  def test_execution(solution: str):
36      code = exec_context.replace("[insert]", solution)
37      for i in range(2):
38          test_input, expected_result = generate_test_case(i + 1)
39          test_env = {"test_input": test_input}
40          exec(code, test_env)
41          assert exec_test(test_env["result"], expected_result)
42
43  try:
44      test_execution('result␣=␣np.percentile_(a,␣p)')  # underscore
           respecte
45      print('SUCCESS_MARKER')
46  except AssertionError:
47      print('TEST_FAILED:␣Assertion␣incorrecte')
48  except Exception as e:
49      print(f'EXEC_ERROR:␣{e}')
```

Listing 2 – Fichier de test DS1000 après injection (perturbation underscore)